Evaluating Four Pairs of AI Models for an Online Retailer

System A Versus System B for Shelf Monitoring, Photo Editing, Super-Resolution and Image Compression

Evaluation report, COMP8410 Assignment 1

Mohammad Rakin Rayan

Student ID: 48658057

Master of Information Technology (Artificial Intelligence)

School of Computing, Macquarie University

COMP8410, Session 2, 2026

Executive Summary

I evaluated two AI systems for an Australian retailer with 100 stores and an online catalogue. Each system has one model for each of four costly processes, so the evaluation compares four pairs: YOLO26s against RF-DETR Nano for shelf monitoring, Qwen-Image-2.1 against FLUX.2 [klein] for photo editing, AuraSR v2 against HYPIR for enlarging supplier photos, and HPCM-Base against DCAE for image compression. Each pair ran in one Kaggle notebook on a T4 GPU, with real data, the same inputs for both models, three seeds where a model is random, paired Wilcoxon tests, measured speed, simple costs, pass or fail limits written into the notebooks before the reported runs (though after a first, larger version with the same models), and my own blind ratings.

Shelf monitoring. RF-DETR Nano was more accurate on all three seeds (F1 0.830 against 0.797; counting error 10.6% against 13.6%), but YOLO26s was 2.5 times faster in my un-optimised timing and recalled more on the nine most crowded photos (too few to test).

Photo editing. FLUX.2 [klein] changed the product less on all four edits where it should stay in place (measured on product pixels), but with so few edits neither this nor the result over all six edits is significant; it was also 4.3 times faster and is Apache 2.0. I rated Qwen-Image-2.1 slightly higher, but not significantly, and its licence is non-commercial.

Super-resolution. I preferred HYPIR on 7 of 7 photos, but AuraSR v2 was more faithful, HYPIR invented more words on average (not significant) and HYPIR's licence is non-commercial.

Compression. Both codecs needed about 81% fewer bits than today’s JPEG on product photos, but no browser can show either, which alone rules them out, and a two-image cross-device check without a CPU-only control also failed; AVIF saves 70% at today's quality and works today.

System B’s models are the ones I would pilot in pairs 1 to 3 (HYPIR only once licensed), but only pair 1’s lead is statistically clear, pair 3’s model needs a licence and pair 4 is a tie, so System B is the better starting point rather than a stack to adopt whole. I recommend RF-DETR Nano after retraining on the store's own photos, FLUX.2 [klein] with a person approving every edit, HYPIR only under a commercial licence with an automatic text and colour check, and AVIF instead of a learned codec. Across the pairs, no single headline metric was enough, licences mattered as much as accuracy, failures compound when the models are chained (Section 7.2), my own quick checks missed errors that the metrics caught, and one of my own metrics (pair 2's product region) needed a correction, which I made from the saved images.

Contents

Executive Summary 2
1 Introduction 4
2 Evaluation Framework 4
2.1 Evaluation Aspects and Metrics 5
2.2 Statistics, Costs and Ratings 6
3 Pair 1: Shelf Monitoring with YOLO26s and RF-DETR Nano 7
3.1 Models and Techniques 7
3.2 Modelling Context 8
3.3 Evaluation Setup 8
3.4 Results 9
3.5 User Feedback 9
3.6 Business, Social and Environmental Performance 10
3.7 Governance, Risk and Trust 10
3.8 Reflection and Recommendation 11
4 Pair 2: Product Photo Editing with Qwen-Image-2.1 and FLUX.2 [klein] 11

4.1 Models and Techniques 11
4.2 Modelling Context 12
4.3 Evaluation Setup 12
4.4 Results 13
4.5 User Feedback 14
4.6 Business, Social and Environmental Performance 15
4.7 Governance, Risk and Trust 15
4.8 Reflection and Recommendation 15
5 Pair 3: Super-Resolution with AuraSR v2 and HYPIR 16
5.1 Models and Techniques 16
5.2 Modelling Context 16
5.3 Evaluation Setup 17
5.4 Results 17
5.5 User Feedback 18
5.6 Business, Social and Environmental Performance 18
5.7 Governance, Risk and Trust 19
5.8 Reflection and Recommendation 19
6 Pair 4: Learned Image Compression with HPCM and DCAE 19
6.1 Models and Techniques 19
6.2 Modelling Context 20
6.3 Evaluation Setup 20
6.4 Results 21
6.5 User Feedback 22
6.6 Business, Social and Environmental Performance 22
6.7 Governance, Risk and Trust 22
6.8 Reflection and Recommendation 22
7 System A Versus System B 23
7.1 Scorecard 23
7.2 How the Models Work Together 24
7.3 What the Four Pairs Show 25
7.4 Validity and Power 26
7.5 Fairness, Safety and Staff 27
7.6 Next Steps 27
8 Conclusion 27
References 29
Appendix A: Implementation Evidence 33
Appendix B: Model Architectures 34

1 Introduction

In this report I act as the chief AI officer of an Australian retailer, called the store, with 100 stores and an online catalogue. Four of its processes are costly: checking shelves (about 8.3 staff hours per store per day if every shelf photo is checked by hand), outsourced photo edits (AUD 17.32 each, about 2,000 a month), reshoots of supplier photos that are too small for the zoom view (AUD 36.09 each, about 1,500 a month), and serving about 50 million product image views a month as JPEG. These volumes are scenario assumptions for a retailer of this size, not measured figures, so each saving in this report scales with them. Each process matches a unit topic, and the store has two candidate systems, A and B, with one model per process (Table 1). The store's objectives are to do each job well below its manual cost, without misleading customers (Competition and Consumer Act 2010 (Cth) sch 2) or collecting personal data it does not need (Privacy Act 1988 (Cth)), and with a named owner for every risk (National Artificial Intelligence Centre [NAIC], 2025).

Table 1
The Four Pairs and the Question Each Pair Answers

Pair and topic	Task	System A	System B	Technical question
1. Computer vision	Detect and count products in shelf photos	YOLO26s: one-stage CNN detector with an NMS-free head	RF-DETR Nano: detection transformer on a DINOv2 backbone	Does a self-supervised transformer beat a small-object CNN when fine-tuned on 500 photos?
2. Image generation and editing	Edit product photos from a text instruction	Qwen-Image-2.1: 7B diffusion transformer trained by flow matching, 40 steps	FLUX.2 [klein] 4B: rectified-flow transformer distilled to 4 steps	Does a larger multi-step model edit more faithfully than a small distilled one?
3. GANs	Enlarge small supplier photos 4 times	AuraSR v2: GigaGAN-style GAN trained from scratch	HYPIR: GAN started from a Stable Diffusion UNet	Does a diffusion prior make a GAN upscaler more faithful, or only more realistic?
4. VAEs	Compress product images for the website	HPCM-Base: hierarchical progressive context model	DCAE: dictionary-based cross-attention entropy model	Which entropy model saves more bits, and can either be deployed?

Note. CNN = convolutional neural network; NMS = non-maximum suppression; GAN = generative adversarial network; VAE = variational autoencoder.

I chose models that are state of the art for the lecture topic (released 2024 to 2026), run on a free 16 GB T4, differ in core technique and could plausibly be deployed. The unit's update of 26 October 2026 reduced the task to four pairs and allowed my own evaluation as user feedback, so I rated every user test myself. Table 2 summarises the results.

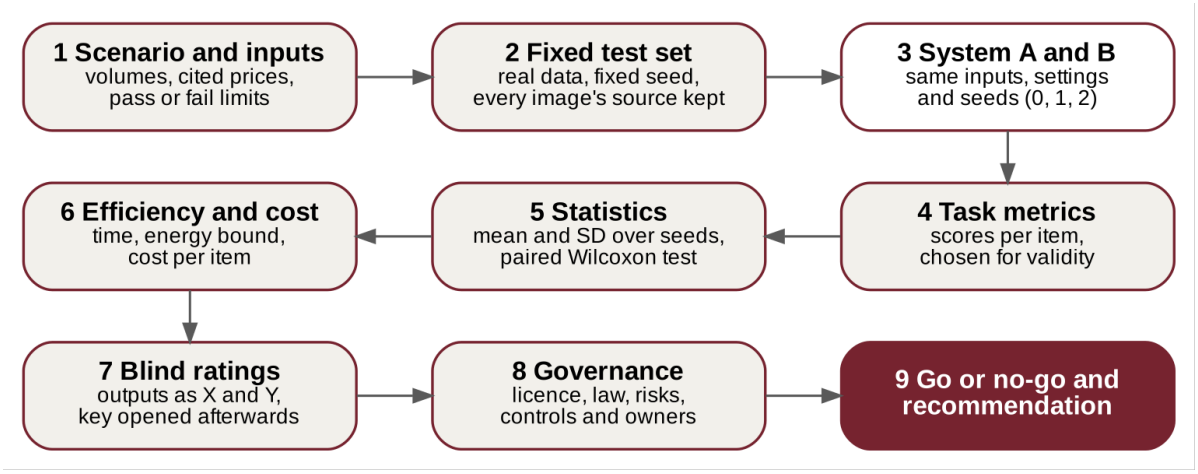
Table 2
Results and Recommendations at a Glance

Pair	What the evidence shows	Recommendation
1 Shelf monitoring	RF-DETR Nano (B) more accurate on all three seeds; YOLO26s (A) 2.5 times faster in my un-optimised timing and recalled more on the nine most crowded photos; neither meets the pilot limits yet	B, after retraining on the store's own photos
2 Photo editing	FLUX.2 [klein] (B) changed the product less on all 4 in-place edits (not significant), is 4.3 times faster and is Apache 2.0; Qwen-Image-2.1 (A) rated slightly higher (not significant) but non-commercial	B with a person approving every edit; sale tags as web overlays
3 Super-resolution	HYPIR (B) preferred on 7 of 7 photos ($p = .016$; .09 after Holm correction) but less faithful than AuraSR v2 (A) and invented more words on average (not significant); B is non-commercial	B only with a commercial licence and an automatic text and colour check
4 Compression	Both about 81% below JPEG and effectively tied with each other; neither shows in a browser, and a cross-device check without a CPU-only control also failed	Neither; switch JPEG to AVIF (70% fewer bits)

2 Evaluation Framework

Every pair follows the same steps in one notebook (Figure 1); only the task metrics change, so results can be compared across pairs and between the two systems.

Figure 1
Evaluation Framework Followed by Every Pair's Notebook



Note. Own diagram. Each box is a section of the notebooks.

2.1 Evaluation Aspects and Metrics

Table 3 maps the aspects of the unit's evaluation methodology to how I measured them. For the task metrics I used the Week 8 distinction between *task validity*, whether the test task represents the real job, and *outcome validity*, whether the score really shows success at it (Y. Zhu et al., 2025), which many published benchmarks fail (Eriksson et al., 2025), and gave each pair one validity check (Table 4).

Table 3
How Each Evaluation Aspect Is Measured

Aspect	How it is measured in every pair
Role and appropriateness against trends	Core technique, lecture link and release date; position among current alternatives from published benchmarks
Technical performance and competitiveness	Task metrics on the same inputs for both models; paired comparison of A and B; baselines (bicubic; JPEG, WebP and AVIF; the papers' own curves)
Business performance	Cost per item from measured GPU and review time, against the manual process at cited prices
Social, economic and environmental effects	Effects on staff, suppliers and customers; GPU energy as an upper bound at the T4's 70 W limit
User feedback	My blind 1 to 5 ratings of the two models' outputs, shown as X and Y
Gaps	Results by shelf density, edit task, input damage and image set
Governance, safety, privacy and ethics	Licence, Australian law and each model's own safety measures; a short risk table with owners
Fairness and trust	Fidelity to the real product (changed products, invented text, colour shifts) and stability across seeds and devices
Interpretation and recommendation	Pass or fail limits per pair; comparison of the two systems (Section 7)

Table 4
Task Metrics and Validity Checks

Pair	Metrics (higher is better unless stated)	Validity check
1 Detection	COCO mAP and AP50; at one working threshold, precision, recall, F1 and counting error (lower)	A stock audit runs at one threshold, chosen on validation photos only; results by shelf density
2 Editing	CLIP directional similarity; on the product only, DINOv2 similarity, LPIPS (lower) and SSIM; OCR of sale tags	Product found from the white background; the notebook scores the rectangle around it, so SSIM was re-checked on product pixels only (Section 4.4)
3 Upscaling	PSNR and SSIM on luma, LPIPS (lower), MUSIQ, CIEDE2000 colour error (lower), OCR word recall and invented words (lower)	Bicubic baseline; MUSIQ read next to fidelity, because it rewards invented texture
4 Compression	Bits per pixel of real files, PSNR, MS-SSIM, BD-rate (lower), speed, the same file decoded on a CPU	My Kodak curves against each paper's published curve

Note. The metrics follow the lectures where possible: SSIM and LPIPS (Z. Wang et al., 2004; Zhang et al., 2018), CLIP direction (Gal et al., 2022), DINO similarity, the identity score of personalised generation, computed here with DINOv2 (Ruiz et al., 2023), MUSIQ (Ke et al., 2021), CIEDE2000 (Sharma et al., 2005) and BD-rate (Bjøntegaard, 2001).

2.2 Statistics, Costs and Ratings

Statistics. Both models process exactly the same items, so every comparison is paired: each model's mean, the share of items on which A is better and a two-sided Wilcoxon signed-rank test (Wilcoxon, 1945). Where a model is random, I ran seeds 0, 1 and 2, report the mean and standard deviation (*SD*) over seeds and average each item over the seeds before testing. With 4 to 10 items the smallest possible *p* ranges from .125 (4 items) to .002 (10 items), so only consistent differences can show, and a high *p* means "not shown" rather than "no difference". Because each pair tests several metrics, I also report *p* after a Holm correction over each pair's main tests, my ratings included (Holm, 1979), bootstrap 95% intervals for mean differences (10,000 resamples) and the matched-pairs rank-biserial correlation *r* as an effect size (+1 when A is higher on every item, -1 when B is), all computed from the saved results by statistics_check.py. With fewer than about 15 items the percentile bootstrap gives intervals that are too narrow, so the intervals for the nine densest shelf photos and the eight product photos are approximate.

Costs. Each notebook measures time per item on the T4 and prices it with the few inputs in Table 5. Energy is an upper bound, the measured time at the T4's 70 W limit.

Ratings. Each notebook saves the two outputs for an item as X and Y in a random order with the key in a separate file, which I opened only after rating. I answered one question per pair on a 1 to 5 scale (for example, "Would I publish this edit?"), timed each item to get a rough review time (most timings were rounded to 5 s), and counted an output as usable when I rated it 4 or 5. Week 8 names disagreement between annotators, one annotator's drift over time and poor reproducibility as the main problems of human evaluation. With one rater and no repeat rating I can measure none of them, and because I built the pipelines I may recognise a model's style even with X and Y hidden, so the ratings show a direction rather than proof and the pilots use staff approval and audits instead.

Platform. All notebooks ran on Kaggle with a Tesla T4 and PyTorch 2.11 on 31 October 2026, with versions and commits recorded in run_info.json (Appendix A).

Table 5
Cost Inputs

Input	Value	Source
GPU time	AUD 0.99 per hour	AWS g4dn.xlarge (one T4), Sydney, on demand, US$0.684 (Amazon Web Services [AWS], n.d.-a)
GPU power	70 W at most	T4 board limit (NVIDIA, 2019)
Outsourced edit; reshoot	AUD 17.32; AUD 36.09	Low end of published prices (Racklify, n.d.; Squareshot, n.d.)
CDN transfer	AUD 0.165 per GB	Amazon CloudFront, Australia and New Zealand, US$0.114 (AWS, n.d.-b)
Store staff; office staff	AUD 31.15; AUD 61.41 per hour	Retail Award level 1 and average weekly earnings, plus superannuation (Australian Bureau of Statistics, 2026; Australian Taxation Office, n.d.; Fair Work Ombudsman, 2026)
Manual shelf-photo check	50.5 s per photo; 595 photos per store per day	My own timing from the first version; 35,000 products per pass and 2.5 passes a day (Supermarket News, 2020) at 147 products per photo, the SKU-110K average (Goldman et al., 2019)
Volumes	100 stores; 2,000 edits, 1,500 photos and 50 million views a month	The store's planning figures

Note. US dollar prices converted at 0.6928 USD per AUD (Exchange-Rates.org, 2026).

3 Pair 1: Shelf Monitoring with YOLO26s and RF-DETR Nano

Key finding. RF-DETR Nano counts products more accurately than YOLO26s on all three seeds, while YOLO26s was 2.5 times faster in my un-optimised timing and recalled more on the nine most crowded photos.

Recommendation: RF-DETR Nano, retrained on the store's own photos before a pilot.

3.1 Models and Techniques

YOLO26s (System A) is Ultralytics' one-stage CNN detector from January 2026 (Sapkota et al., 2025; Ultralytics, 2026). An end-to-end head with one-to-one label assignment removes non-maximum suppression (NMS), small-target-aware label assignment (STAL) and ProgLoss help small objects, and the MuSGD optimiser steadies training. The small model has 9.9 million parameters (48.6 COCO mAP at 2.5 ms on a T4) and is AGPL-3.0, with a paid licence for closed use.

RF-DETR Nano (System B), from Roboflow (ICLR 2026), is a real-time detection transformer (Robinson et al., 2026). Learned object queries attend to the image through a deformable-attention decoder (X. Zhu et al., 2021), its DINOv2 backbone is pretrained without labels (Oquab et al., 2024), and its layout is chosen by weight-sharing architecture search. The Nano model has 30.5 million parameters (48.0 COCO AP in the paper, 48.4 in the current repository, at 2.3 ms) and is Apache 2.0.

Technique. YOLO regresses boxes densely from the whole image (Redmon et al., 2016). YOLO26 decides which prediction learns each object by task-aligned assignment: every candidate gets the score $t = s^{\alpha} \cdot u^{\beta}$, where s is its predicted class score, u its box overlap (IoU) with the object and α and β set their balance (Feng et al., 2021). During training a one-to-many head keeps the top few candidates per object for a richer signal, while the one-to-one head used at inference keeps only the best one, so no NMS is needed (A. Wang et al., 2024); STAL adds a candidate for objects too small to receive one. DETR instead treats detection as set prediction and pairs predictions with objects one to one through the Hungarian algorithm (Equation 1), so duplicates never appear (Carion et al., 2020). Both detectors therefore end with one prediction per object, so the pair mainly tests the backbone: a CNN tuned for small objects against a self-supervised transformer. Shelf photos are a hard test, with about 143 small products per photo against about 7 objects in a COCO image (T.-Y. Lin et al., 2014). One-to-one matching also has a cost here: each query can claim only one product, so a fixed budget of 300 queries has to stretch across the densest shelves. Figure B1 in Appendix B shows where each technique acts in the two pipelines.

$$\hat{\sigma} = \arg\min_{\sigma \in S_N} \sum_{i=1}^{N} \mathcal{L}_{\text{match}}\left(y_i,\ \hat{y}_{\sigma(i)}\right) \tag{1}$$

Here each y_i is a ground-truth object, sigma ranges over the ways of pairing them with the N predictions, and the matching cost combines class agreement and box overlap.

3.2 Modelling Context

Table 6
Modelling Context of Pair 1

Aspect	YOLO26s (System A)	RF-DETR Nano (System B)
Objectives	Business: replace the manual check of shelf photos with automatic counts and gap flags. Technical: high F1 and low counting error at one threshold, under 200 ms per photo. *(Same for both systems)*	
Task	Single-class detection of densely packed products, then a count per photo. *(Same for both systems)*	
Functions	Dense prediction at every location; each product predicted once	Up to 300 object queries decoded into a set of boxes
Framework	CNN backbone, feature-pyramid neck, end-to-end head (ultralytics 8.4.189)	DINOv2 backbone, multi-scale projector, deformable decoder (rfdetr 1.11.2)
Algorithms	Task-aligned assignment with STAL, ProgLoss, MuSGD, NMS-free inference	Self-supervised pretraining, architecture search, Hungarian matching
Governance	AGPL-3.0 or a paid licence; the documentation lists no safety or privacy features	Apache 2.0; the documentation lists no safety or privacy features
Advantages	Fast; easy to export to CPUs and edge devices	Transfers well to new domains through DINOv2
Known failure modes (my test)	Lower recall overall; reached its 300-box cap on the densest photo	Lower recall on the nine most crowded photos (not significant)
Training data	COCO-pretrained weights (T.-Y. Lin et al., 2014)	DINOv2 backbone pretrained without labels on LVD-142M (Oquab et al., 2024), then COCO

Table 6 sets out the modelling context. Neither model documents its own safety measures, so the controls in Table 8 carry the risk. On COCO the two sit among other real-time detectors such as D-FINE-S (50.6 AP) and LW-DETR-S (48.0 AP) (Robinson et al., 2026); D-FINE is also Apache 2.0 (Roboflow, 2026), which makes it the natural next candidate (running further models was outside this assignment's four-pair scope). The two are a fair pair on their published figures (about 48 COCO AP at about 2.4 ms on a T4 with TensorRT), which is how a practitioner shortlists detectors, although RF-DETR Nano has three times the parameters. COCO is a weak guide here: the two models are 0.2 to 0.6 AP apart on COCO but 3.3 F1 points apart on shelves. In my end-to-end timing (PyTorch at full precision, including reading the photo and keeping up to 300 boxes) RF-DETR Nano took 55.8 ms a photo against 21.9 ms for YOLO26s, but its library warned that the model was not in its optimised FP16 inference mode (train_log.txt), so this gap reflects my deployment path rather than the models; their published T4 TensorRT latencies are 2.3 and 2.5 ms. Both are well inside my 200 ms limit (about 4 and 9 times faster than it), so speed does not decide this pair.

3.3 Evaluation Setup

I used SKU-110K (Goldman et al., 2019) through a Kaggle copy in YOLO label format (thedatasith, n.d.), with a fixed random split (seed 0) of 500 training, 100 validation and 150 test photos; the test photos hold 21,443 products, 143 per photo on average (66 to 358). Both detectors were fine-tuned from COCO weights on the same photos for 12 epochs at 640 px, capped at 300 detections per photo, with seeds 0, 1 and 2. Each used its library's recipe at the largest batch that fits a T4 (8 for YOLO26s; 4 with 4 accumulation steps, an effective 16, for RF-DETR Nano), so the comparison is between practical recipes rather than identical ones. I report COCO mAP and AP50, and, at the threshold with the best F1 on the validation photos, precision, recall, F1 and the counting error (the absolute difference between found and true counts over the true count), each averaged over photos, plus milliseconds per photo. My pilot limits were F1 of at least 0.85, counting error of at most 10% and under 200 ms per photo.

3.4 Results

Table 7

Pair 1 Results on 150 Test Photos, Mean (SD) Over Three Seeds

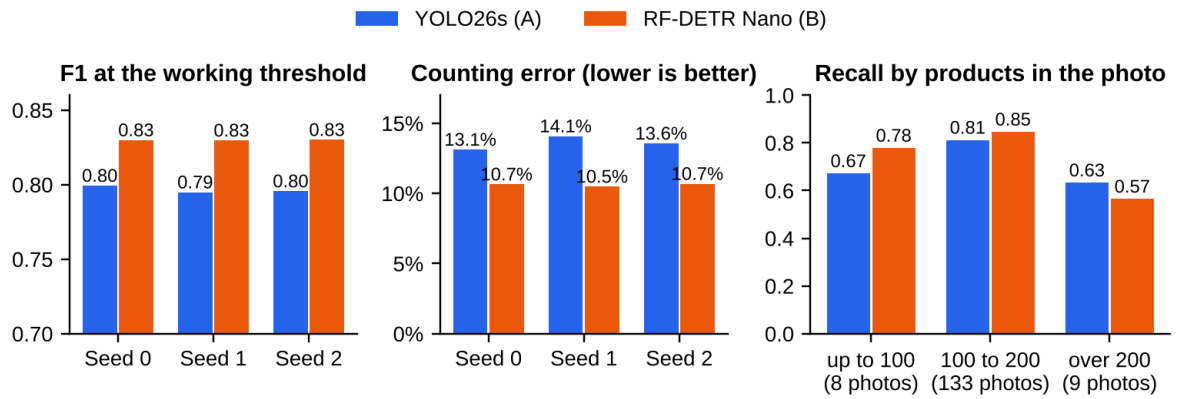
Measure	YOLO26s (A)	RF-DETR Nano (B)	Photos where A is better	p
Precision	0.817 (0.0020)	0.843 (0.0003)	27%	< .001
Recall	0.794 (0.0048)	0.826 (0.0004)	22%	< .001
F1	0.797 (0.0024)	0.830 (0.0002)	19%	< .001
Counting error (lower is better)	13.6% (0.48 points)	10.6% (0.10 points)	33%	< .001
COCO mAP; AP50	0.358; 0.816	0.372; 0.836		
Working threshold	0.25 to 0.30	0.40 on all seeds		
Time per photo; training per seed	21.9 ms; 3 min	55.8 ms; 12 min		
Pilot limits met (F1, count, speed)	No, no, yes	No, no, yes		

Note. The paired test compares the 150 photos, each averaged over the seeds. After a Holm correction across the five tests, my ratings included, all four measures stay below .001. Mean difference, A minus B, with bootstrap 95% interval: F1 -0.033 (-0.044 to -0.023, r = -0.69); counting error +3.0 percentage points (1.5 to 4.5, r = 0.44). Times are end to end in PyTorch at full precision, with RF-DETR Nano not in its optimised inference mode (Section 3.2).

RF-DETR Nano was better on every accuracy measure and every seed (Table 7, Figure 2), with the higher F1 on 122 of the 150 photos. It was also more stable: its threshold was 0.40 on every seed and the standard deviation of its F1 over seeds was 0.0002, against 0.0024 for YOLO26s. Recall (by 0.032) and precision (by 0.026) both contributed to the gap. The exception was the nine photos with more than 200 products, where YOLO26s recalled more (0.633 against 0.565), but only on 5 of the 9 and not significantly (difference 0.068, 95% interval -0.021 to 0.164, p = .36), so the reversal is not shown; on the 8 photos with up to 100 products RF-DETR Nano was better on 7 of 8 (p = .039, uncorrected). On the densest photo (358 products, seed 0) YOLO26s drew 300 boxes, its cap, of which 268 matched a product, while RF-DETR Nano drew 227, of which only 146 matched. RF-DETR Nano kept at most 229 boxes above its threshold on these photos, well under its 300-query cap, so the cap alone does not explain the gap; low confidence on crowded shelves may, and nine photos are too few to tell. COCO mAP ranked the models the same way as F1 but understated the gap at the working threshold (0.014 against 0.033), because it averages over strict overlaps that matter little for counting.

Figure 2

Pair 1 Accuracy by Seed and Recall by Shelf Density



Note. F1 and counting error for each training seed (the F1 axis starts at 0.70); recall by the number of products in the photo, averaged over seeds. Data: the 150 SKU-110K test photos (Goldman et al., 2019).

3.5 User Feedback

I rated ten test photos with both detectors' boxes drawn in the same colour. RF-DETR Nano averaged 4.6 (90% rated 4 or 5) and YOLO26s 3.5 (50%); RF-DETR Nano scored higher on seven photos, YOLO26s on one, with two ties (p = .070). After opening the key, I compared my scores with each photo's F1 from the seed 0 models, which drew the boxes I rated: my preference matched the metric on five of the eight photos where I had one, and all three mismatches favoured RF-DETR Nano on photos where it had missed many products, including a 5 on the densest photo. Across all 20 rated outputs my

score was unrelated to that photo's F1 (Spearman's rho = 0.01; the 20 outputs come from ten photos, so they are not independent). Wrong boxes are easy to see; missing boxes on a busy shelf are not, so a reviewer would miss the under-counting that hides real gaps.

3.6 Business, Social and Environmental Performance

If all 595 of a store's shelf photos were checked by hand, it would take about 8.3 staff hours a day (AUD 260), using my 50.5 s per photo from one timing session in the first version, for which I kept no log, so this baseline is an upper bound. While rating, I also timed how long each photo took me to judge with the boxes drawn: about 9 s on average, but most timings were rounded to 5 s, so the figure is rough. The notebook first described this timing as the manual-check time; I use it as the time to review the model's boxes, because I was judging boxes rather than counting products. At about 9 s a photo, review takes about 1.5 hours (about AUD 46), so either model would save up to about AUD 214 per store per day, or AUD 21,400 a day across 100 stores.

Because the 50.5 s has no log, I checked how much the saving depends on it (saving_checks.py): at 40 s a photo the saving would be about AUD 160 per store per day, and at half the time (25 s) about AUD 83. It disappears only if checking a photo by hand took no longer than the 9 s review, so the case for a pilot does not rest on the exact timing. Both timings should be re-measured in the pilot, and because a quick review misses under-counting (Section 3.5), the pilot also hand-counts a sample every month (Table 8). Even RF-DETR Nano misses about 28 products on an average test photo (about one product in five; the mean recall per photo is 0.83, but recall is lowest on the fullest shelves; saving_checks.py), so the counts guide staff to likely gaps rather than replace a check. GPU cost and energy are negligible: RF-DETR Nano needs 33 s of GPU time per store per day, under one cent and at most 0.65 Wh. The saving therefore depends on accuracy, through how much checking the counts need, not on speed; the cameras that take the photos are costed below as a break-even. Staff would confirm flagged gaps instead of scanning shelves, so the store must consult them first (General Retail Industry Award 2020 cl 34).

These figures assume that every shelf photo would otherwise be checked by hand. Without cameras, gaps are found by walking the aisles, so a second baseline is a full manual audit: a mid-sized store stocks 10,000 to 20,000 products, and Simbe Robotics' chief executive has put auditing them at 20 to 30 staff hours a week (Kelly, 2015), about AUD 620 to 930 a week at the Retail Award rate. That is a vendor's estimate for a smaller range than the store's 35,000 products, so it understates the store's audit. Covering the store once a week from photos needs about 238 photos (35,000 ÷ 147) and about 35 minutes of box review (AUD 18), so at today's frequency the labour saving is about AUD 600 to 915 per store per week, or roughly AUD 2,600 to 4,000 a month. That is the most the cameras or robots that take the photos can cost per store before the labour case fails. Photographing 2.5 times a day instead buys more frequent gap checks, a benefit for availability rather than labour, which the pilot should measure.

3.7 Governance, Risk and Trust

Table 8

Pair 1 Risks and Mitigations

Risk	Mitigation	Owner
YOLO26s is AGPL-3.0	No closed use without an Enterprise licence; RF-DETR Nano is Apache 2.0	Head of legal
SKU-110K is research-only data	Retrain on the store's own labelled photos before going live	ML platform lead
Shoppers and staff appear in photos (Privacy Act 1988, APPs 3 and 11; the OAIC's 2024 Bunnings finding)	Photograph outside trading hours or blur faces; delete photos within 7 days	Privacy officer
Missed products look like gaps and cause wrong reorders	Staff confirm every flagged gap; monthly F1 on 50 hand-counted photos, retrain below 0.85	Store operations manager

Note. APP = Australian Privacy Principle; OAIC = Office of the Australian Information Commissioner (Commissioner Initiated Investigation into Bunnings Group Ltd (Privacy), 2024).

Table 8 lists the main risks. The detectors decide nothing about people. The fairness risk is between products: a category with small or unusual packaging may be under-counted and run out more often, so

counting error should be tracked by category. The same applies to shelf density: on the photos with more than 200 products both models under-counted by about a quarter (YOLO26s 22%, RF-DETR Nano 26%, as (found minus true) over true), against 2 to 3% overall, and across all 150 photos under-counting grew with the number of products (Spearman's rho = -0.39 for YOLO26s and -0.55 for RF-DETR Nano, both $p < .001$), so false gap flags will cluster on the fullest shelves, which is why the pilot reports them separately. RF-DETR Nano's stable threshold across seeds also makes it easier to monitor and trust.

3.8 Reflection and Recommendation

Recommendation. Adopt RF-DETR Nano (System B) after retraining on about 500 of the store's own photos and a pilot in a few stores, reporting the most crowded shelves separately. It is close to, but not at, my limits (F1 0.830 against 0.85; counting error 10.6% against 10%). YOLO26s suits edge devices and may suit very crowded shelves (it recalled more on the nine most crowded photos, too few to test) but needs a paid licence.

Interpretation. At this data size RF-DETR Nano, with its self-supervised DINOv2 backbone, transferred better than a CNN tuned for small objects, at the cost of speed in my un-optimised timing. It is also three times larger (30.5 million against 9.9 million parameters) and was trained with a different recipe, so this test cannot say how much of the gain comes from the backbone.

Gaps and future work. The training budget was small (500 photos at 640 px), SKU-110K has no labels for empty facings, so gap detection itself was not measured, and my ratings showed that review alone misses under-counting. Training at higher resolution or on tiles, lowering the threshold or raising the query budget for crowded shelves, adding planogram data and testing D-FINE-S are the next steps. I would switch to YOLO26s if the pilot showed a significant recall gap on dense shelves or required edge devices, and if a paid licence is acceptable.

4 Pair 2: Product Photo Editing with Qwen-Image-2.1 and FLUX.2 [klein]

Key finding. FLUX.2 [klein] changed the product less than Qwen-Image-2.1 on all four edits where it should stay in place (measured on product pixels). Four edits cannot give a significant result, and I grouped them after the runs, and over all six non-colour edits the difference is not significant either. Klein is also 4.3 times faster and licensed for commercial use, and neither model writes sale tags reliably. **Recommendation:** FLUX.2 [klein] for studio backgrounds and, on only two edits, colour variants, with a person approving every edit.

4.1 Models and Techniques

Qwen-Image-2.1 (System A), released by Alibaba's Qwen team in September 2026, is a latent model whose 7-billion-parameter diffusion transformer is conditioned by the Qwen3-VL 8B vision-language model, which reads the instruction together with the photo (AI Weekly, 2026; Qwen Team, 2026). Qwen-Image was trained by flow matching (Qwen Team, 2025), and version 2.1's pipeline samples with the same flow-matching Euler scheduler (Hugging Face, 2026). Its recommended setting is 40 steps without guidance, and its licence is now the non-commercial Qwen Research License. To fit a 16 GB T4 I quantised it to 4-bit NF4 (Dettmers et al., 2023) and used CPU offload.

FLUX.2 [klein] 4B (System B), from Black Forest Labs (January 2026), is a 4-billion-parameter rectified-flow transformer for generation and editing, step-distilled so that 4 steps without guidance replace the usual few dozen (Black Forest Labs, 2026). It is Apache 2.0, and its model card describes filtered training data, red teaming, content filters and C2PA (Coalition for Content Provenance and Authenticity) provenance.

Technique. Both editors are latent flow-matching transformers (Rombach et al., 2022; Peebles & Xie, 2023), the velocity-field counterpart of the noise prediction that diffusion models learn (Ho et al., 2020). They learn a velocity field that carries noise to an image along the straight path between the two (Equation 2) (Lipman et al., 2023; Liu et al., 2023), and an edit is produced by following that field from noise to image in a fixed number of Euler steps (Equation 3). The pair therefore contrasts scale and sampling within one family rather than diffusion with flow: Qwen-Image-2.1 takes 40 steps with a 7B

transformer that reads the photo and the instruction through Qwen3-VL, while klein is step-distilled so that 4 large steps reproduce its teacher (Black Forest Labs, 2026; Figure B2 in Appendix B). Both models take the photo as a condition, so identity preservation has to be learned rather than enforced. Neither used classifier-free guidance here (Qwen-Image-2.1's recommended setting and klein's distilled one), so the strength of an edit was not tuned at inference; distillation also tends to make outputs vary less between seeds, which helps consistency but limits choice.

$$x_t = (1 - t)\, x_0 + t\, \varepsilon, \qquad \mathcal{L}_{\mathrm{RF}} = \mathbb{E}_{x_0,\, \varepsilon,\, t} \| v_\theta(x_t,\ t,\ c) - (\varepsilon - x_0) \|^2 \tag{2}$$

$$x_{t - \Delta} = x_t - \Delta\, v_\theta(x_t,\ t,\ c), \qquad \Delta = 1/N \tag{3}$$

Here x_0 is the clean latent, ε the noise, c the instruction and photo, v_θ the predicted velocity and N the number of steps, 40 for Qwen-Image-2.1 and 4 for klein.

4.2 Modelling Context

Table 9
Modelling Context of Pair 2

Aspect	Qwen-Image-2.1 (System A)	FLUX.2 [klein] 4B (System B)
Objectives	Business: replace outsourced retouching (AUD 17.32 an edit) without misrepresenting products. Technical: follow the instruction while leaving the product's shape, print and colour unchanged, within 60 s. *(Same for both systems)*	
Task	Instruction-based editing: a photo and a text instruction in, an edited photo out. *(Same for both systems)*	
Functions	Studio background, lifestyle background, colour variant and sale tag	
Framework	Qwen3-VL encoder; 7B transformer, 40 steps; VAE decoder	Text and image encoding; 4B flow transformer, 4 steps; VAE decoder; here float16 with CPU offload
Algorithms	Flow matching, 40 steps without guidance; here NF4 quantisation and CPU offload	Rectified flow (flow matching) step-distilled to 4 steps, no guidance
Governance	Non-commercial research licence; no safety statement	Apache 2.0; data filtering, red teaming, filters, C2PA
Advantages	Strong multimodal reading of the instruction	Small and fast, made for consumer GPUs

Table 9 sets out the modelling context. Neither model has been scored on a public editing benchmark yet. On GEdit-Bench-EN the strongest open editor, Qwen-Image-Edit (7.56), matches the closed GPT Image 1 (7.53), with Step1X-Edit at 6.97 (Qwen Team, 2025); both open models are Apache 2.0 and worth testing next.

4.3 Evaluation Setup

Four catalogue photos on white backgrounds were picked at random from Amazon Berkeley Objects (ABO; CC BY 4.0) (Collins et al., 2022): a stylus pen, a backpack, a power bank and a USB hub. Each got two of four tasks (studio background, lifestyle background, navy colour variant, "SALE 20% OFF" tag), giving 8 edits per model, each with seeds 0, 1 and 2 and the same photo, instruction and seed for both models. Instructions were templated from the ABO titles, which named the pen "devices pink" and the hub "adapter renewed"; both models got the same wording, which may have made some edits worse for both. I measured CLIP directional similarity for instruction following; DINOv2 similarity, LPIPS and SSIM for the product in the six edits that should not change it; OCR of the sale tags with EasyOCR (JaidedAI, n.d.); and seconds per edit. The notebook finds the product from the white background but scores the rectangle around it, and for the pen that rectangle covers 96% of the image and is only 17% pen, so a changed background also lowers the score. I therefore re-scored SSIM on the product pixels only, from the saved images, with a short script (product_pixel_check.py), and report both. My limits were product SSIM of at least 0.70, OCR match of at least 0.90 and at most 60 s per edit.

4.4 Results

Table 10

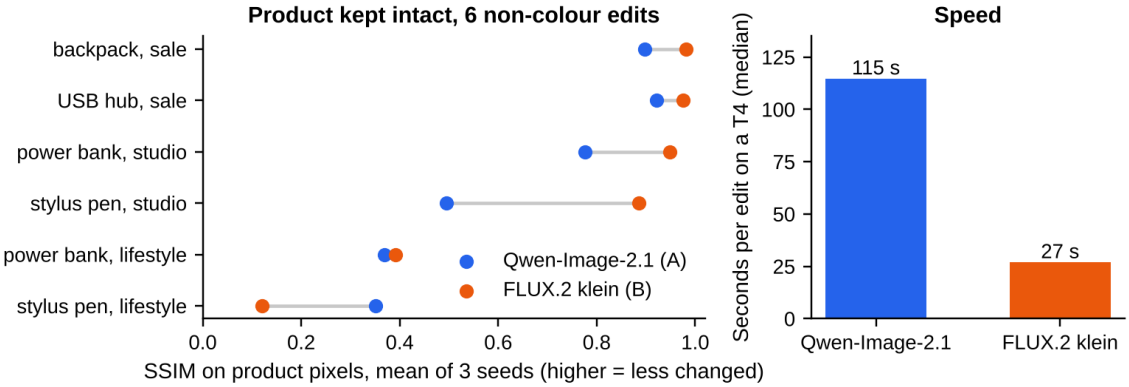
Pair 2 Results, Mean Over Three Seeds

Measure	Qwen-Image-2.1 (A)	FLUX.2 [klein] (B)	Edits where A is better	p
CLIP direction, 8 edits	0.129	0.115	6 of 8	.148
SSIM, rectangle around the product, 6 edits	0.738	0.804	0 of 6	.031
SSIM, product pixels only, 6 edits	0.635	0.717	1 of 6	.31
SSIM, product pixels, 4 in-place edits	0.773	0.948	0 of 4	.125
DINOv2 similarity (rectangle), 6 edits	0.874	0.828	2 of 6	1.0
LPIPS (rectangle, lower is better), 6 edits	0.301	0.299	3 of 6	1.0
Sale tags containing the phrase; OCR match	2 of 6; 0.58	3 of 6; 0.62		
Seconds per edit (T4, median)	114.7	26.9		
Pilot limits met (SSIM, OCR, speed)	Yes (no on product pixels), no, no	Yes, no, yes		

Note. Paired tests compare edits averaged over the three seeds; colour edits are left out because they are meant to change the product. In-place edits are the studio backgrounds and sale tags. Product-pixel SSIM comes from product_pixel_check.py; the other product scores are the notebook's rectangle scores. Exact OCR matches: Qwen-Image-2.1 1, klein 2. Pair 2's tests have 4 to 8 items and its in-place group was chosen after the runs, so no bootstrap interval is given; after a Holm correction no difference in this table is significant.

The editors were close on instruction following: Qwen-Image-2.1 moved the image further in the direction the text asked on 6 of 8 edits, but not significantly (Table 10). On the notebook's rectangle around the product, klein scored a higher SSIM on all six non-colour edits, but for the pen and the power bank that rectangle covers almost the whole image (96% and 98%), so this partly measures how much the background changed, an outcome-validity problem I found when I checked what the rectangle contained. On the product pixels alone (Figure 3), klein was still higher on five of six edits (0.717 against 0.635), but not significantly (p = .31). On the four edits where the product should stay in place, the studio backgrounds and sale tags, klein changed the product less on all four (0.948 against 0.773). That is the most consistent result four items allow, but it cannot be significant (p = .125), and I grouped these edits after the runs, so it is a direction for the pilot to confirm rather than a proven difference. It was consistent, though: klein's product-pixel SSIM was higher in all 12 seed-level runs of those four edits (runs of one edit are not independent, so this shows consistency, not significance). The notebook's rectangle result also does not survive a Holm correction across the pair's tests (p = .16). In the lifestyle scenes both models moved or resized the product (Figure 4), so pixel scores cannot measure preservation there; klein's lifestyle pen scored lowest of all because it moved the pen into the room. Klein was also more consistent across seeds (product-pixel SSIM SD per edit 0.016 against 0.130). On the two colour edits, which the preservation scores leave out because they are meant to change the product, the models were close: CLIP direction 0.188 for Qwen-Image-2.1 and 0.178 for klein, rectangle SSIM 0.824 and 0.791, and DINOv2 similarity 0.945 and 0.953 (by_task.csv); I rated klein's two colour edits 4 and 4 and Qwen-Image-2.1's 5 and 3. Sale tags left the product intact but failed on the text: two of Qwen-Image-2.1's six tags and three of klein's contained the phrase, and the OCR metric was too strict, because it read the hub's printed logo and misread a correct tag that klein had drawn at an angle.

Figure 3
Product Preservation per Edit and Speed in Pair 2



Note. Left: SSIM on the product pixels only for each non-colour edit, mean of three seeds (product_pixel_check.py). Right: median seconds per edit on a T4.

Figure 4
Example Edits From Both Models (Seed 0)



Note. Input photos from Amazon Berkeley Objects (Collins et al., 2022; CC BY 4.0).

4.5 User Feedback

I rated the eight seed-0 edits next to their inputs. Qwen-Image-2.1 averaged 4.12 (5 of 8 publishable) and klein 3.50 (4 of 8); Qwen-Image-2.1 was higher on five edits (p = .42). I preferred Qwen-Image-2.1 on both studio backgrounds (5 and 4 against klein's 3 and 3) even though it had changed the product more (seed 0 product-pixel SSIM 0.56 and 0.55 against klein's 0.84 and 0.94), and on the lifestyle power bank, because its scenes looked more natural. On those two studio edits klein also had its lowest CLIP direction of any task (−0.017 against 0.024 for Qwen-Image-2.1; by_task.csv): it kept the product but barely changed the scene. A reviewer rewards realism, which is why approval must be made against the original photo. This bears on the recommendation: in the scope I recommend, studio backgrounds and colour variants, klein's publishable share is 2 of 4, the same half as overall, but both were colour variants and neither studio background passed, so the editing pilot's approval gate is a real test of klein's studio backgrounds rather than a formality. Each decision took me about 18 s (rough timings, mostly rounded to 5 s).

4.6 Business, Social and Environmental Performance

An edit costs about AUD 0.31 with klein (GPU AUD 0.007 and review AUD 0.30) or AUD 0.33 with Qwen-Image-2.1, against AUD 17.32 from the agency. At 2,000 edits a month, counting only the edits I would publish, klein saves about AUD 17,000 a month and Qwen-Image-2.1 about AUD 21,200 (about AUD 250 to 300 less if the GPU and review time of rejected edits is also counted), but Qwen-Image-2.1 cannot be used commercially and at 115 s an edit it fails my 60 s limit. These savings rest on eight ratings: with 4 of 8 klein edits publishable, the 95% Wilson interval for that share is 22% to 78%, so klein's saving, with every attempt charged, could be anywhere from about AUD 6,800 to 26,600 a month (saving_checks.py). The pilot's 200 edits would narrow the share to about 7 points either side. Even at the lower end of that interval, the saving repays the AUD 12,000 that both pilots cost (Section 7.6) in under two months. Energy is small, about 1.0 kWh a month for klein and 4.5 kWh for Qwen-Image-2.1. The agency's volume would fall, and the content team would approve edits instead of ordering them.

4.7 Governance, Risk and Trust

Table 11
Pair 2 Risks and Mitigations

Risk	Mitigation	Owner
An edit changes the product's shape, logo or colour (ACL ss 18 and 29; Trade Marks Act 1995 (Cth) s 120)	A person approves every edit; product-pixel SSIM below 0.70 rejected automatically	Content team lead
Generated sale text or prices are wrong (ACL s 29(1)(i))	Prices and sale tags only as web overlays	E-commerce manager
Qwen-Image-2.1 is licensed for non-commercial use only	No deployment without a written commercial licence	Head of legal
Customers cannot tell an image was edited	Keep the original; add C2PA credentials to every edit (C2PA, n.d.)	Content team lead
Off-brand or harmful output (open models have few built-in filters)	Prompts only from fixed templates; every output reviewed	Brand manager

Note. ACL = Australian Consumer Law, schedule 2 of the Competition and Consumer Act 2010 (Cth).

Table 11 lists the main risks. My test had no photos of people, so edits should stay on product photos until a separate check shows no change to skin tone or body shape. Trust here means that the published photo shows the real product, which only approval and automatic checks can guarantee.

4.8 Reflection and Recommendation

Recommendation. Adopt FLUX.2 [klein] (System B) for studio backgrounds and, on the evidence of only two edits, colour variants, with a person approving every edit and sale tags as web overlays; keep lifestyle backgrounds out of scope for now. The studio-background scope is a test rather than a finding: klein kept the product intact there partly by barely changing the scene (CLIP direction −0.017), and I rated neither of its two studio backgrounds publishable (Section 4.5), so a high preservation score must never pass an edit on its own. The pilot therefore passes an edit only when the approver confirms that the requested change is visible next to the original and the product-pixel SSIM is at least 0.70. If fewer than half of the studio backgrounds pass, the pilot narrows to colour variants and tests a paste-back pipeline instead, which generates the background and then restores the original product pixels, so the product is preserved by construction.

Interpretation. The larger, slower model did not edit more faithfully: ten times the steps bought a small, uncertain gain in instruction following and realism, while the distilled model changed the product less where it should stay in place and was more consistent, which matters more to a retailer. Qwen-Image-2.1 had to run in 4-bit, which may have cost it some quality, so this shows that the small model is good enough rather than that size does not help.

Gaps and future work. Eight edits is a small test, each task was tried on only two products, Qwen-Image-2.1 ran 4-bit with offload, and the OCR check should be limited to the tag region. My first preservation score partly measured the background, the outcome-validity failure Week 8 warns about, so a masked score should be the default, and lifestyle edits need a measure that does not depend on

alignment, such as DINOv2 on the segmented product. A pipeline that pastes the original product pixels back after generating the background would guarantee preservation and is the natural next candidate, alongside Qwen-Image-Edit and Step1X-Edit. Qwen-Image-2.1 would only come back into consideration with a commercial licence and an edit under 60 s.

5 Pair 3: Super-Resolution with AuraSR v2 and HYPIR

Key finding. I preferred HYPIR's enlargements on all seven photos I rated, but AuraSR v2 is more faithful to the real photo, HYPIR invented more words on average (not significant over six inputs), and HYPIR's licence is non-commercial.

Recommendation: neither as tested; HYPIR only with a commercial licence and an automatic text and colour check.

5.1 Models and Techniques

AuraSR v2 (System A), from fal (2024; 0.62 billion parameters), is a GAN upscaler based on GigaGAN's image-conditioned upsampler (Kang et al., 2023). It enlarges 4 times in one pass over 64-pixel tiles, with an offset second pass to hide seams, and its model card presents it as an upscaler for generated images. The weights are Apache 2.0.

HYPIR (System B) starts its generator from Stable Diffusion 2.1's UNet and fine-tunes it adversarially with low-rank adapters (LoRA, rank 256; Hu et al., 2022), then restores an image in one forward pass at a fixed noise level, guided by a text prompt that names the product (X. Lin et al., 2025). Its licence forbids processing images intended for sale (SupPixel Pty Ltd, 2024), and Stable Diffusion 2.1 is no longer on its official repository, so the notebook loads a community copy (Manojb/stable-diffusion-2-1-base, pinned to revision 0094d48; geogeo28, 2025). Stable Diffusion 2.1 was trained on a filtered subset of LAION-5B and carries the CreativeML Open RAIL++-M licence, whose use restrictions also apply to HYPIR's outputs (Stability AI, 2022).

Technique. A GAN trains a generator against a discriminator (Equation 4) (Goodfellow et al., 2014). For super-resolution an adversarial loss gives sharp, realistic texture at the cost of PSNR (Ledig et al., 2017). HYPIR keeps the adversarial loss but starts from a diffusion model's weights, linking the GANs of Week 9 with the diffusion priors of Week 10, so the pair asks whether that prior makes enlargements more faithful or only more realistic (Blau & Michaeli, 2018). Restoring in one step at a fixed noise level, guided by a text prompt, also means the prior fills in missing detail from what such products usually look like, which is how plausible but wrong lettering can appear. Figure B3 in Appendix B compares the two pipelines.

$$\min_{G} \max_{D} \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log (1 - D(G(z)))] \tag{4}$$

Here G maps the small photo to an enlargement and D scores how real an image looks, which rewards realistic texture whether or not it was in the original.

5.2 Modelling Context

Table 12
Modelling Context of Pair 3

Aspect	AuraSR v2 (System A)	HYPIR (System B)
Objectives	Business: avoid reshoots (AUD 36.09 each) of supplier photos too small for the zoom view. Technical: enlarge 4 times, faithful to the product and natural-looking. *(Same for both systems)*	
Task	Single-image super-resolution of real, degraded product photos. *(Same for both systems)*	
Functions	4 times upscaling in overlapping tiles	Restoration and 4 times upscaling, guided by the product name
Framework	GigaGAN-style generator, one pass per tile (aura-sr 0.0.4)	SD 2.1 UNet with LoRA at t = 200; VAE decoder (commit b61d107)
Algorithms	Adversarial training from scratch	Diffusion-initialised adversarial fine-tuning
Governance	Apache 2.0 weights; no safety features	Non-commercial licence; base model removed from its official repository

Aspect	AuraSR v2 (System A)	HYPIR (System B)
Advantages	Fast, small and permissively licensed	Realistic restoration of real damage in one step

Table 12 sets out the modelling context. In its authors' DIV2K comparison HYPIR (MUSIQ 72.6) matches the multi-step SeeSR (72.7) in one pass and beats Real-ESRGAN (69.4), the permissively licensed alternative (X. Lin et al., 2025; X. Wang et al., 2021). MUSIQ, however, does not compare with the original, so it rewards the invented detail a retailer must avoid.

5.3 Evaluation Setup

Eight ABO photos (Collins et al., 2022) at 1024 px were shrunk 4 times to 256 px in two ways, clean (bicubic) and supplier-like (blur, then JPEG quality 60), giving 16 inputs with a known real photo; three photos carry printed text. Both models and a bicubic baseline enlarged every input with seeds 0, 1 and 2. HYPIR had to run in float32, because float16 gave black images on the T4, which slows it down, and both models turned out to be effectively deterministic, so the seeds changed almost nothing. I measured PSNR and SSIM on luma, LPIPS, MUSIQ, the CIEDE2000 colour error on the product, OCR word recall and invented words on the photos with text, and seconds per photo. My limits were LPIPS of at most 0.10, colour error no more than 1.0 above bicubic, at most 0.5 invented words per photo and at most 30 s per photo.

5.4 Results

Table 13
Pair 3 Results on 16 Inputs

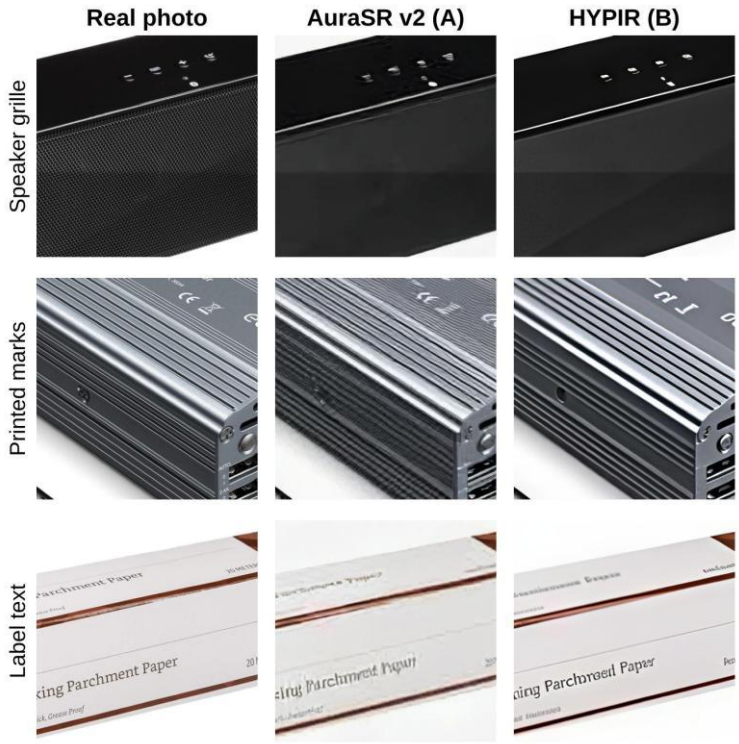
Measure	AuraSR v2 (A)	HYPIR (B)	Bicubic	Inputs where A is better	p
PSNR on luma (dB)	26.52	25.03	27.38	15 of 16	< .001
SSIM on luma	0.881	0.853	0.847	13 of 16	.001
LPIPS (lower is better)	0.117	0.116	0.278	6 of 16	.74
MUSIQ (realism)	69.3	72.9	46.4	5 of 16	.034
Colour error, CIEDE2000 (lower)	4.45	5.08	4.12	13 of 16	.018
Word recall, 6 inputs with text	0.116	0.136	0.022	1 of 6	1.0
Invented words per photo with text (lower)	1.28	2.11	0.67	3 of 6	.25
Seconds per photo (T4, median)	4.6	12.1			
Pilot limits met (LPIPS, colour, text, speed)	No, yes, no, yes	No, yes, no, yes			

Note. Paired tests use each input's mean over seeds; the seeds changed mean PSNR by less than 0.01 dB (at most 0.02 dB on any input). After a Holm correction across the eight tests, my ratings included, PSNR ($p < .001$) and SSIM ($p = .009$) stay significant, while MUSIQ (.13) and colour error (.09) do not. PSNR difference, A minus B, +1.49 dB (bootstrap 95% interval 1.05 to 1.95, $r = 0.99$).

The two models sit on opposite sides of the perception-distortion trade-off (Table 13). AuraSR v2 was more faithful, with higher PSNR on 15 of 16 inputs, on both clean ($p = .016$) and supplier-like inputs ($p = .008$), and higher SSIM; its smaller colour shift and HYPIR's higher MUSIQ point the same way but do not survive a Holm correction ($p = .09$ and .13). HYPIR's MUSIQ advantage came from the supplier-like inputs (5.9 points, $p = .039$, 8 inputs) rather than the clean ones (1.2 points, $p = .46$), and the two tied on LPIPS. Bicubic had the best PSNR of all and by far the worst realism, so PSNR alone would have picked the blurry output. HYPIR was also more robust: on the supplier-like inputs AuraSR v2's MUSIQ fell from 72.1 to 66.5 as it sharpened the JPEG blocks, while HYPIR held at 72.3. Both models invented words, HYPIR more on average (2.1 against 1.3 per photo, not significant over six inputs), and even bicubic produced 0.7 OCR "inventions" from blur alone, so the count has a noise floor and its limit should be set relative to bicubic. On the baking paper HYPIR turned "Parchment Paper" into "Parchroed Paper" (Figure 5).

Figure 5

Crops From Three Supplier-Like Inputs: Real Photo, AuraSR v2 and HYPIR



Note. Same 384 px crop from the real photo and each enlargement (seed 0). Photos from Amazon Berkeley Objects (Collins et al., 2022; CC BY 4.0).

5.5 User Feedback

I rated seven supplier-like enlargements next to the real photo (the white easel's crop was blank and was left out). HYPIR averaged 4.43 (6 of 7 publishable) and AuraSR v2 2.14 (none), with HYPIR higher on all seven (p = .016). My quick check, about 7 s a photo, missed what the metrics found: I gave "Parchroed Paper" a 5 and redrawn CE marks a 4, and did not mark down either model for smoothing away a speaker grille. My ratings measured how good an enlargement looks, not whether it is true to the product: across the 14 rated outputs my score rose with MUSIQ (Spearman's rho = 0.53, p = .052, approximate because the 14 outputs come from seven photos) but not with LPIPS (rho = -0.16) or colour error (rho = 0.20), and after a Holm correction my preference itself is no longer significant (p = .09).

5.6 Business, Social and Environmental Performance

An enlargement costs about AUD 0.13 including a check of about 7 s, against AUD 36.09 for a reshoot. At 1,500 photos a month, counting only the photos I would publish, HYPIR would save about AUD 46,000 a month if licensed, while AuraSR v2 saves nothing because none of its enlargements was publishable. This saving is the most uncertain figure in the report, since it rests on seven ratings, in which I passed an invented word, and an unknown licence fee. With 6 of 7 enlargements publishable, the 95% Wilson interval for that share is 49% to 97%, so even before the text gate and the licence fee the saving could be as low as about AUD 26,000 (saving_checks.py). Energy is at most 0.35 kWh a month. Small suppliers, who send the weakest photos, would gain most and also carry most of the risk of invented detail. On invented words alone, my own text gate (Table 14) would have passed HYPIR on one of the six inputs with text (none once the colour condition is added) and AuraSR v2 on two. Applied in full to the seven rated enlargements, it passes only two of the six I rated publishable: all three with text fail, and so does one without. Counting the rest as reshoots leaves a saving of about AUD 15,000 a month (saving_checks.py), a third of the figure above, until the gate or the noise level changes. Because the licence fee is unknown, the decision rule is a ceiling: with the gate applied, HYPIR stays worthwhile at this volume only while its licence costs less than about AUD 15,000 a month.

5.7 Governance, Risk and Trust

Table 14
Pair 3 Risks and Mitigations

Risk	Mitigation	Owner
HYPIR's licence forbids images intended for sale	No HYPIR output published without a signed commercial licence	Head of legal
Invented text, logos or parts mislead customers (ACL ss 18 and 29; Trade Marks Act 1995 (Cth) s 120)	Automatic gate: OCR finds no more invented words than on the bicubic enlargement of the same photo, colour error no more than 1.0 above bicubic's; then a person checks	Content team lead
Suppliers own their photos (Copyright Act 1968)	Supplier terms must allow enlargement	Supplier relations manager
Stable Diffusion 2.1 has been removed from its official repository	Mirror it internally, pinned by revision and SHA-256	ML platform lead

Table 14 lists the main risks. The more natural an enlargement looks, the harder an invented detail is to see, so trust here depends on automatic checks rather than visual review, and on asking suppliers for larger originals first.

5.8 Reflection and Recommendation

Recommendation. Deploy neither model as tested. Pilot HYPIR (System B) only after a commercial licence and behind the automatic text and colour gate; until then, ask suppliers for larger originals.

Interpretation. HYPIR, with its diffusion prior, was more realistic and more robust to real damage but not more faithful, and its realistic texture came with plausible but wrong words. The models also differ in size, training data and target domain (AuraSR v2 was made for generated images), so this test cannot isolate the prior.

Gaps and future work. The damage was simulated from good photos, eight photos and one rater limit the test, and OCR only finds invented words where there is text, with a noise floor even on bicubic. My 7-of-7 preference gives the smallest p that seven items allow (.016), but it ranked third of the pair's eight tests, so the Holm correction multiplied it by six; nine consistent ratings (p = .004) would survive the correction wherever they ranked (.031), which sets the minimum size of a repeat rating. Testing Real-ESRGAN as a licensable baseline, a lower HYPIR noise level and real supplier photos are the next steps. HYPIR would become the recommendation once it has a commercial licence and passes the text gate on at least five of six photos with text.

6 Pair 4: Learned Image Compression with HPCM and DCAE

Key finding. Both codecs reproduce their papers within 1% and need about 81% fewer bits than today’s JPEG on product photos, but neither shows in a browser, which alone rules them out, and a two-image cross-device check without a CPU-only control also failed, while AVIF already saves 70%.

Recommendation: switch the website from JPEG to AVIF and revisit learned codecs when browsers support one.

6.1 Models and Techniques

Both are learned transform codecs built as variational autoencoders (Ballé et al., 2018; Kingma & Welling, 2014). An encoder maps the image to a latent that is quantised and entropy-coded, a hyperprior predicts the scale of each latent, a context model refines each latent's probability from those already decoded (Minnen et al., 2018), and a decoder rebuilds the image. Training minimises bits plus lambda times distortion (Equation 5), which up to constants is the negative ELBO of a VAE with a uniform-noise posterior (Ballé et al., 2018); one checkpoint is trained per quality level. **HPCM-Base (System A)** (Li et al., 2025; ICCV 2025; 68.5 million parameters) codes latents across scales with a hierarchical schedule and progressive context fusion.

DCAE (System B) (Lu et al., 2025; CVPR 2025; 119.4 million parameters) adds a learned dictionary of typical structures that the entropy model queries by cross-attention. Both are MIT licensed. The context

model is also the technique's weak point for deployment: each latent's probability depends on latents already decoded, so decoding is partly serial, and the encoder and decoder must compute bit-identical probabilities, which floating-point differences between a GPU and a CPU can break. Figure B4 in Appendix B shows the shared design and where the two entropy models act.

$$\mathcal{L} = \mathbb{E}_x\left[-\log_2 p(\hat{y} \mid \hat{z}) - \log_2 p(\hat{z})\right] \; + \; \lambda \, \mathbb{E}_x \|x - \hat{x}\|^2 \tag{5}$$

The first term counts the bits for the latent and hyperprior, the second the squared error, and lambda sets the trade-off. The pair contrasts context from inside the image (HPCM) with a prior learned from the training set (DCAE), and tests whether a benchmark result carries over to product photos with large plain backgrounds.

6.2 Modelling Context

Table 15
Modelling Context of Pair 4

Aspect	HPCM-Base (System A)	DCAE (System B)
Objectives	Business: cut CDN cost (AUD 736 a month with JPEG) and page weight without a visible change. Technical: fewest bits at equal quality, with fast decoding that gives the same image on every device. *(Same for both systems)*	
Task	Lossy image compression: encode once on the server, decode on every viewer's device. *(Same for both systems)*	
Functions	Encode an image at one of four quality levels and decode it back.	
Framework	VAE transforms, hyperprior, hierarchical progressive context model (commit c271cfe)	Same transforms with a dictionary cross-attention entropy model (commit e2525a0)
Algorithms	Rate-distortion training (an ELBO), hierarchical scheduling	Rate-distortion training, learned dictionary
Governance	MIT; trained on Flickr2W; 274 MB pickled checkpoints	MIT; trained on OpenImages; 1,434 MB pickled checkpoints
Advantages	Strong compression at lower complexity	A data-driven prior for typical structures

Table 15 sets out the modelling context. Among published learned codecs on Kodak, HPCM-Base (15.3% fewer bits than the VTM reference) and DCAE (16.8%) sit within about 1.5 points of each other, with larger models such as HPCM-Large (19.2%) slightly ahead (Li et al., 2025; Xiong et al., 2026). The practical competitor is AVIF, which browsers already display, so I measured it directly.

6.3 Evaluation Setup

I used eight ABO product photos (Collins et al., 2022) at 1024 px and the 24 Kodak images (Franzen, n.d.), the benchmark both papers use. Each codec ran its four MSE checkpoints from the authors' code at fixed commits, with SHA-256 fingerprints recorded, and JPEG, WebP and AVIF ran at nine quality levels; every bitstream was written as a real file. I measured bits per pixel, PSNR and MS-SSIM (Z. Wang et al., 2003), BD-rate at equal PSNR (Bjøntegaard, 2001), encode and decode time, and whether a file encoded on the GPU decodes to the same image on a CPU. As a validity check I compared my Kodak curves with each paper's. My limits were at least 10% fewer bits than AVIF, a CPU decode within 0.1 dB of the GPU and display in a browser.

6.4 Results

Table 16

Pair 4 BD-Rates (Negative Means Fewer Bits)

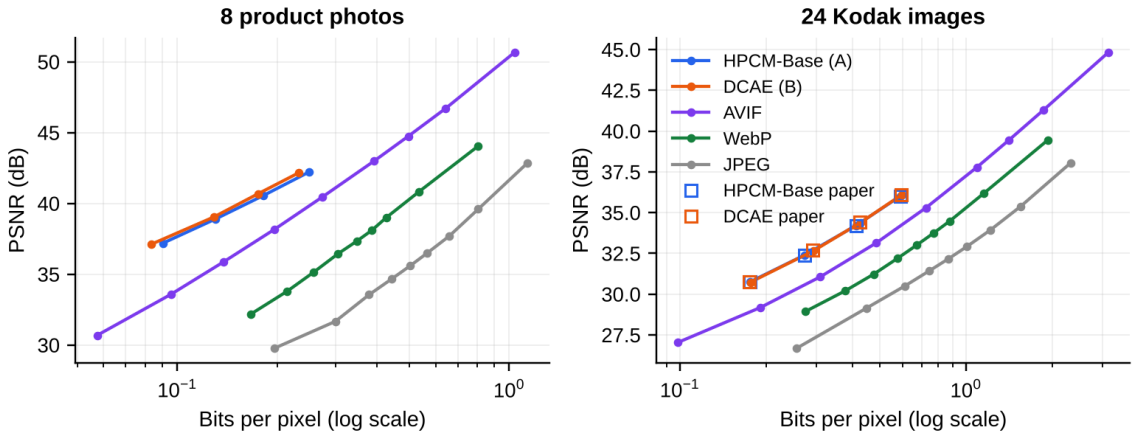
Comparison	Product photos (8)	Kodak (24)
HPCM-Base; DCAE against JPEG	-81.3%; -82.4%	-68.9%; -68.8%
HPCM-Base; DCAE against AVIF	-37.8%; -41.2%	-32.8%; -32.5%
AVIF against JPEG	-72.6%	-55.2%
HPCM-Base against DCAE, pooled PSNR curves	+5.4%	-0.3%
HPCM-Base against DCAE, pooled MS-SSIM curves	+9.9%	+2.4%
HPCM-Base against DCAE, per image: mean; better on; p	+0.4%; 4 of 8; .84	-1.1%; 20 of 24; < .001
My curve against the paper's curve (HPCM-Base; DCAE)		+0.1%; +0.9%

Note. BD-rates are at equal PSNR unless the row says MS-SSIM. Per-image values are paired over images; pooled values compare one curve per codec averaged over the set. Per-image HPCM-Base against DCAE, bootstrap 95% interval: Kodak -1.1% (-1.6 to -0.6, r = -0.75, Holm p = .002); products +0.4% (-9.2 to +9.5), which supports calling them tied.

My Kodak curves matched both papers within 1% (Table 16, Figure 6), so the pipeline and bit counting are valid. On the product photos both codecs needed about 81% fewer bits than JPEG and 38 to 41% fewer than AVIF on the pooled curves (per product photo, HPCM-Base 36% on average, bootstrap 95% interval 18 to 50%, and DCAE 36%, 18 to 51%), more than on Kodak. I expected this to come from the plain backgrounds, but across the eight photos the share of near-white pixels did not track the saving against AVIF (Spearman's rho = -0.02 for HPCM-Base and 0.12 for DCAE), so the reason is not shown. Between the two codecs the answer depends on how it is measured: HPCM-Base was better per image on Kodak (1.1%, 20 of 24 images) but only 0.3% on the pooled curves, while DCAE was ahead on the pooled product curves (5.4%) and on MS-SSIM for both sets (9.9% and 2.4%), and per product photo the difference swung from 27% in HPCM-Base's favour to 25% against it. In practice they are tied, so a difference of about 1% on a benchmark should not decide anything. My limit of beating AVIF by at least 10% held on 7 of 8 product photos for HPCM-Base and 6 of 8 for DCAE; on one photo HPCM-Base needed 18% more bits than AVIF, and on another DCAE needed 11% more. Deployment is where both failed (Table 17): no browser can display either format, which alone rules them out, and in my two-image test a file encoded on the T4 and decoded on a CPU came out at 4 to 13 dB, an unusable image. Entropy decoding needs bit-identical probabilities on both devices, which floating-point differences can break (Ballé et al., 2019), but without a CPU-only control a fault in the CPU code path cannot be ruled out.

Figure 6

Rate-Distortion Curves on the Product Photos and on Kodak



Note. PSNR against bits per pixel (log scale). Open squares are each paper's published Kodak points (Li et al., 2025; Lu et al., 2025).

Table 17
Deployability at Today's Quality (JPEG Quality 80 on the Product Photos)

Format	Bits per pixel	Fewer bytes than today	CDN per month (AUD)	Decode	In browsers	Same image on a CPU
JPEG 80 (today)	0.807		736	3 ms (CPU)	Yes	Yes
WebP	0.458	43%	418	6 ms (CPU)	Yes	Yes
AVIF	0.242	70%	221	8 ms (CPU)	Yes	Yes
HPCM-Base	0.150	81%	137	0.54 s (GPU)	No	No (4 to 5 dB)
DCAE	0.144	82%	131	0.99 s (GPU)	No	No (7 to 13 dB)

Note. Decode times are medians over the eight product photos, from rd_points.csv (speed_cost.csv gives medians over all images); CDN cost at 50 million views a month. A CPU decode of a learned file also took 11 to 13 s. CPU decoding was tested on two product photos at lambda 0.0067, with no CPU-only control.

6.5 User Feedback

I compared crops of seven product photos decoded at about 0.13 bpp (84% fewer bits than today's JPEG) with the original at full zoom. Both codecs averaged above 4 (DCAE 4.43, HPCM-Base 4.29), six of seven crops each scored 4 or 5, and four of seven were ties ($p = 1.0$). At this rate the two look the same, so the choice between them would rest on speed and size, where HPCM-Base is about 1.7 times faster to encode and decode a product photo (1.8 times to decode), has about 43% fewer parameters and has checkpoint files five times smaller.

6.6 Business, Social and Environmental Performance

At 50 million image views a month, JPEG costs about AUD 736 in CDN transfer. AVIF cuts that to AUD 221, saving AUD 515 a month, and makes an average image arrive in 21 ms instead of 71 ms on a slow 10 Mbit/s connection. A learned codec would save only a further AUD 84 to 90 a month, and only if customers could decode it, while moving a heavy neural decoder onto every phone, which costs energy and excludes older devices.

6.7 Governance, Risk and Trust

Table 18
Pair 4 Risks and Mitigations

Risk	Mitigation	Owner
A file decodes to a corrupted product image on another device	Cross-device decode within 0.1 dB before any rollout	Web platform lead
Checkpoints are pickled files that can run code	Pin SHA-256 hashes; load only in a sandbox	Security lead
Files open only with one model version (lock-in)	Keep JPEG or AVIF masters as the archive	Web platform lead
Training images used under varying licences	Legal check of the training data terms before commercial use	Head of legal

Table 18 lists the main risks. No model here decides about people, but a learned codec would shift cost and risk onto customers with older phones, and trust depends on every device decoding the same picture. AVIF is displayed by all current major browsers (Can I use, n.d.).

6.8 Reflection and Recommendation

Recommendation. Deploy neither codec; switch to AVIF with a JPEG fallback. If a learned codec is reconsidered, start from HPCM-Base for its speed and size.

Interpretation. Both entropy models work and beat AVIF by about 40% on product photos, but the difference between them is small, depends on the measure and is smaller than the variation between product photos, and neither addresses what blocks deployment: identical, fast decoding in a browser.

Gaps and future work. Eight product photos is a small set, only MSE checkpoints were tested, and the cross-device test used two photos without a CPU-encode, CPU-decode control, so a bug in the CPU code path cannot be ruled out as the cause. My ratings compared the two learned codecs with each other, not with AVIF, the format I recommend, and AVIF was matched to today's JPEG on PSNR only, although its lead holds at equal MS-SSIM (67% fewer bits than JPEG at equal MS-SSIM on the product

photos), so the AVIF step should include a side-by-side visual check at full zoom before rollout. JPEG AI, the first learned image coding standard (International Organization for Standardization [ISO], 2025), is the likely route into browsers, and integer networks would fix cross-device decoding (Ballé et al., 2019). A learned codec should be reconsidered only when a browser can decode it, a CPU-only control passes and it beats AVIF by 10% on most of the store's own images.

7 System A Versus System B

7.1 Scorecard

Table 19 brings the pairs together. System B gives the better result in pair 1, a better but not significant result on in-place edits in pair 2 and the preferred output in pair 3, while the two codecs in pair 4 are effectively tied and neither should be used. System A is faster in three pairs (in pair 1 only in my un-optimised timing), and System B has three models licensed for closed commercial use against System A's two. At the scenario's volumes either system's GPU time costs under AUD 80 a month (about AUD 47 for System B in pairs 1 to 3), far below the manual work it targets, so GPU cost never decided a pair; accuracy, review effort and licences did.

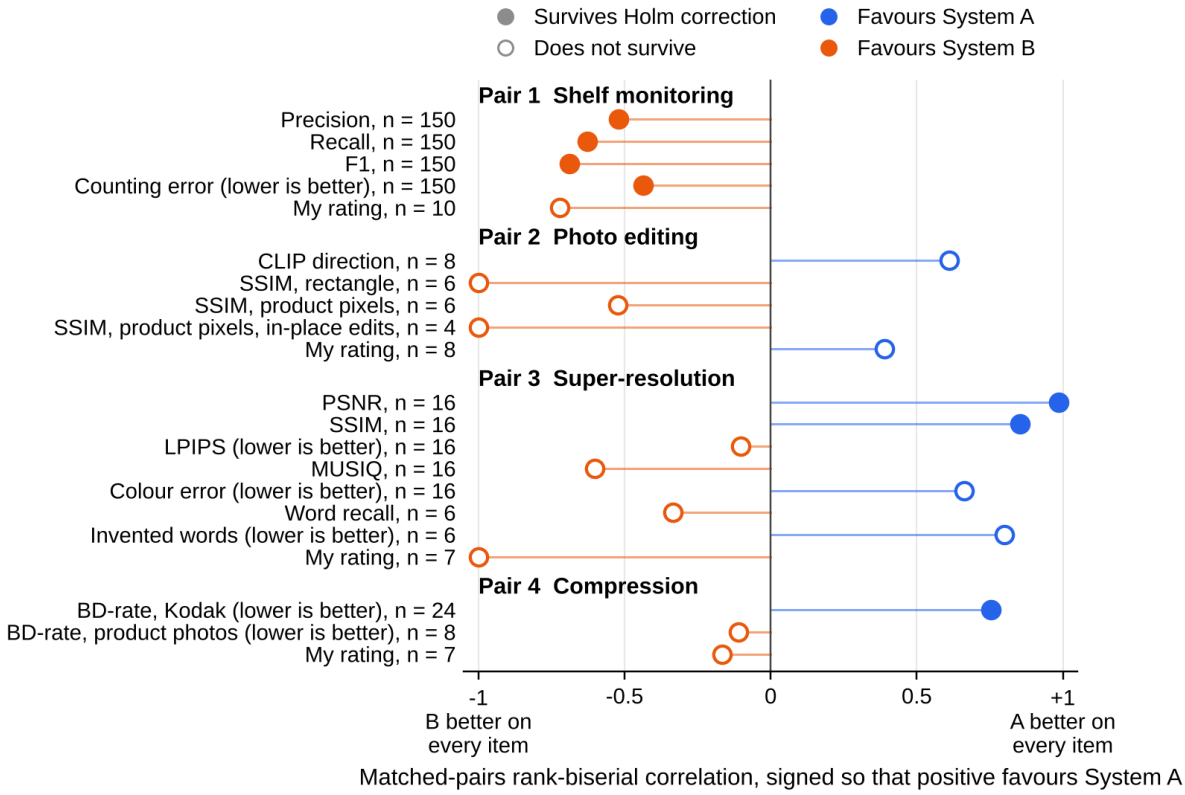
Table 19
System A Against System B, Pair by Pair

Pair	Better task result	Faster	My ratings	Commercial licence (A / B)	Recommended
1. Shelf monitoring	B on F1 and counting error (A recalled more on the 9 most crowded photos)	A (2.5 times, un-optimised)	B (7 of 10)	AGPL or paid / Apache 2.0	B, after retraining
2. Photo editing	B on in-place edits (4 of 4, not significant)	B (4.3 times)	A (5 of 8, not significant)	Research only / Apache 2.0	B, with approval
3. Super-resolution	A on fidelity; B on realism and robustness	A (2.6 times)	B (7 of 7)	Apache 2.0 / non-commercial	B, only if licensed
4. Compression	Tied: A per image on Kodak, B on MS-SSIM and pooled product curves	A (about 1.7 times)	Tie	MIT / MIT	Neither: AVIF

Figure 7 puts every paired comparison on one scale, the matched-pairs rank-biserial correlation, signed so that positive values favour System A. Only pair 1's four measures, pair 3's PSNR and SSIM and pair 4's Kodak BD-rate survive a Holm correction. Most of the other effects are large but rest on 4 to 8 items; Section 7.4 turns them into the sample sizes a decision would need.

Figure 7

Effect Size of Every Paired Comparison, Signed So That Positive Favours System A

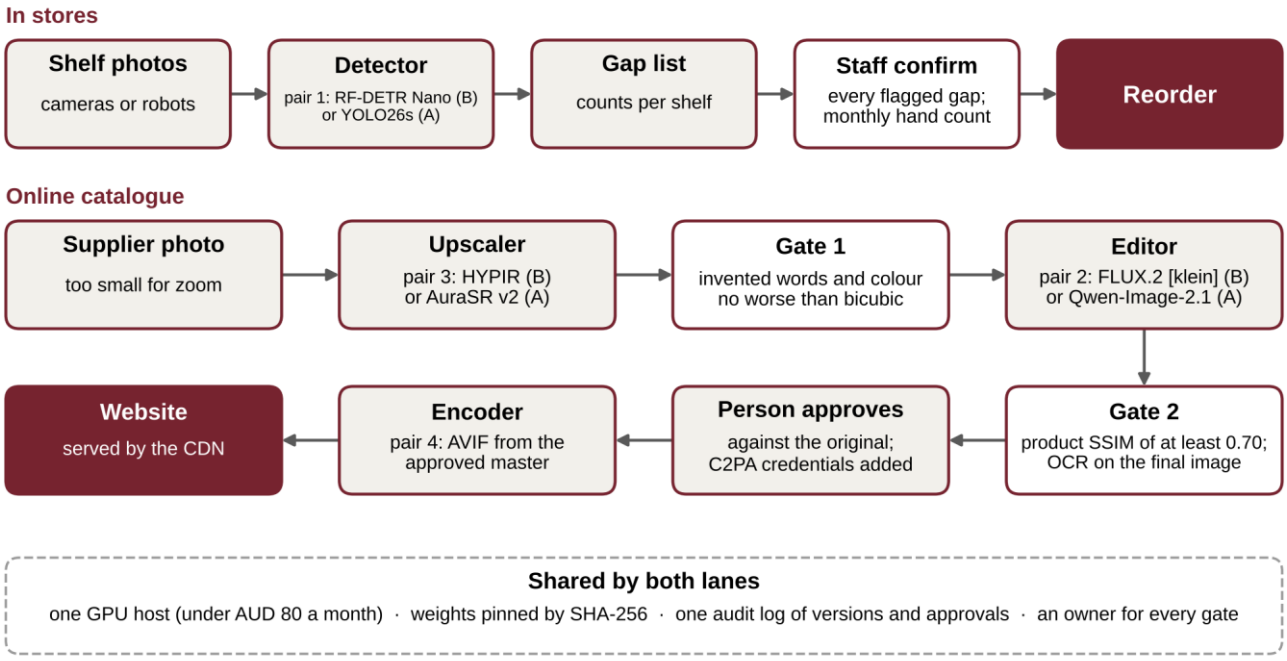


Note. Matched-pairs rank-biserial correlation from statistics_check.csv, with the sign reversed for measures where lower is better; filled markers survive a Holm correction within their pair. My ratings are included.

7.2 How the Models Work Together

Each system runs as one pipeline with two lanes (Figure 8). In stores, shelf photos go to the detector, its counts become a gap list, and staff confirm every flagged gap before anything is reordered. Online, a supplier photo that is too small is enlarged, checked automatically for invented text and colour shifts, edited, checked again on the product pixels, approved by a person with C2PA credentials attached, and only then encoded for the website from the approved master. Three things follow from this order that the pair-by-pair results do not show.

Figure 8
How the Four Models Work Together in Each System



Note. Own diagram. White boxes are automatic gates or checks by staff, with the thresholds of Tables 8, 11 and 14; dark boxes are where each lane ends.

Gates must also run on the final image. The editor is built to keep the product unchanged, so it also keeps anything the upscaler invented, such as "Parchroed Paper"; the OCR check therefore runs again on the final image, not only after enlargement.

Failures compound. Only 2 of the 7 HYPIR enlargements I rated were both publishable and passed the text and colour gate (saving_checks.py), and 4 of klein's 8 edits were publishable. If the two steps were independent, only about 14% of the supplier photos that need both would pass without rework, so for these photos the system saving is far below the sum of the pair savings, and a person approves once at the end rather than after every step.

Compression comes last. Encoding is lossy, so enlarging or editing an already compressed file would compound its artefacts; the store keeps approved masters and encodes AVIF from them (Table 18).

The systems also differ as wholes. System A needs two commercial agreements before it can run end to end (YOLO26s for closed use and Qwen-Image-2.1 for any commercial use), while System B needs one (HYPIR); until that licence is signed, System B's online lane skips enlargement and asks suppliers for larger originals (Section 5.8). Either system would share one GPU host (under AUD 80 a month, Section 7.1), one model registry that pins weights by SHA-256 as pair 4 did, and one audit log of model versions and approvals, each with an owner from the risk tables.

7.3 What the Four Pairs Show

No headline metric was enough. mAP understated the shelf result, PSNR preferred blurry bicubic, MUSIQ rewarded invented detail, rate-distortion preferred codecs no browser can decode, and my own rectangle-based preservation score partly measured the background. Each decision needed a metric at the operating point, a check of fidelity to the product and a deployability test, which is the Week 8 distinction between task and outcome validity in practice.

The four ways an evaluation goes wrong all appeared (Eriksson et al., 2025). Noisy or biased data: SKU-110K has no labels for empty facings, and every pair 2 product sat on a plain white background. Not practical: pair 4's rate-distortion curves ignore browser support and cross-device decoding. Can be gamed: pair 2's rectangle score rose when the background changed less, and MUSIQ rises with invented texture. Not hard enough: pair 3's damage was simulated from good photos. The pilot gates in

Section 7.6 address these with the store's own photos, the most crowded shelves, browser tests and product audits.

Models built on pretrained priors did better, and the priors also invent. RF-DETR Nano, built on DINOv2, learned shelves from 500 photos better than a CNN tuned for small objects (though it is also three times larger and was trained with a different recipe), and HYPIR, built on Stable Diffusion, gave far more natural enlargements than a GAN trained from scratch (I preferred it on 7 of 7, though its MUSIQ lead does not survive a Holm correction); the same kind of prior wrote "Parchroed Paper". For a retailer, whose images are claims about products, realism without fidelity is a legal risk.

A small distilled model held its own, and licences mattered as much as accuracy. FLUX.2 [klein] changed products less on the in-place edits (4 of 4, not significant) than a model with nearly twice its parameters and ten times its steps, at 4.3 times the speed; Qwen-Image-2.1 ran in 4-bit, so this shows that the small model is good enough rather than that size does not help. Qwen-Image-2.1 and HYPIR cannot be used commercially without an agreement and YOLO26s needs a paid licence for closed use, so licences should be checked before models are tested.

My own checks were the weakest evidence. I trusted boxes that missed many products and passed enlargements that fail my own text and colour gate, so the controls that protect customers here are automatic, with people confirming rather than detecting.

7.4 Validity and Power

Table 20 collects, for each pair, the confounds named in its section, the direction in which each would bias the comparison, the control that would remove it, and the smallest difference the test could detect. Pair 1 and the fidelity scores of pair 3 are well powered; pair 2's preservation result, pair 3's realism and text results and pair 4's product photos are not, so a high *p* there means a test too small to decide rather than no difference, and the pilot sizes in Section 7.6 follow from these numbers.

Table 20
Confounds, Controls and Statistical Power by Pair

Pair	Main confounds and the direction of their bias	Control that would remove them	Smallest detectable difference (80% power)
1 Shelf monitoring	12 epochs with each library's recipe: RF-DETR Nano's validation mAP was still rising at epoch 11 (train_log.txt) and YOLO26s's curve was not logged (likely favours B, which adapts faster from its prior); three times the parameters (favours B); timed without inference optimisation (favours A on speed)	Learning curves or one 50-epoch run; a matched effective batch; both models timed in FP16 or TensorRT	F1: 0.015 at 150 photos (observed 0.033); about 32 photos would show the observed gap
2 Photo editing	Qwen-Image-2.1 in 4-bit with CPU offload (handicaps A in quality and speed); templated instructions with odd product names (both); no check that the background changed, so preservation can reward edits that barely change the scene (favours B)	A full-precision run through a hosted API; human-written instructions; a background-change score next to product SSIM	Product-pixel SSIM: 0.23 at 6 edits (observed 0.08); about 49 edits needed
3 Super-resolution	Damage simulated from good photos; AuraSR v2 made for generated images (handicaps A); HYPIR in float32 (slows B); one rater who built the pipelines	Real supplier photos; Real-ESRGAN as a licensable third model; a second rater	PSNR: 0.66 dB at 16 inputs (observed 1.49); MUSIQ needs about 23 inputs, colour error about 22 and invented words about 25 photos with text
4 Compression	MSE checkpoints only (favours PSNR over MS-SSIM); cross-device test without a CPU-only control; 8 product photos	MS-SSIM checkpoints; a CPU-encode, CPU-decode control; the store's own photos	BD-rate: 0.75% on 24 Kodak images (observed 1.1%) but about 14.5% on 8 product photos; about 68 photos would resolve a 5% difference

Note. Smallest detectable difference = $(z_{.975} + z_{.80}) \times SD / \sqrt{n}$, and items needed = $((z_{.975} + z_{.80}) \times SD$ / difference$)^2$: a normal approximation for a paired comparison with 80% power and two-sided α = .05 (Cohen, 1988), where *SD* is the standard deviation of the per-item differences in per_photo.csv, product_pixel_ssim.csv, scores.csv and bd_per_image.csv. The Wilcoxon test needs slightly more items, and standard deviations from 6 to 16 items are rough.

The same calculation applies to monitoring. The monthly audit of 50 hand-counted shelf photos in Table 8 estimates F1 only to within about ±0.04, because RF-DETR Nano's F1 varies from photo to photo with a standard deviation of 0.137; that is wider than the 0.02 gap between its 0.830 and the 0.85 limit. The audit can catch a large drop but cannot confirm the limit, so the shelf pilot's acceptance test uses about 180 hand-counted photos, which narrows the estimate to about ±0.02.

7.5 Fairness, Safety and Staff

No model in either system decides about individuals, so group fairness metrics do not apply; the risks are misleading product images (pairs 2 and 3), the privacy of people in store photos (pair 1) and the integrity of delivered images (pair 4), with uneven effects on product categories, small suppliers and customers with older phones. Only FLUX.2 [klein] documents its own safety measures, so for the other models the controls in the risk tables are the store's, each with an owner, in line with the Guidance for AI Adoption (NAIC, 2025). Shelf monitoring and editing change staff work rather than remove it, so the store must consult staff before a pilot, as the Fair Work Act 2009 (Cth) and the Retail Award require.

7.6 Next Steps

I recommend a staged rollout, where each step continues only if its gate is met:

1. **Licences and privacy first.** Exclude YOLO26s and Qwen-Image-2.1 from production, ask SupPixel about a HYPIR licence, and complete a privacy assessment for shelf photos.
2. **AVIF now.** Re-encode catalogue images with a JPEG fallback; gate: at least 60% fewer bytes at today's quality and no broken images in browser tests.
3. **Editing pilot.** FLUX.2 [klein] for studio backgrounds and, on only two edits, colour variants, with approval of both the requested change and the unchanged product; gate: at least half of 200 pilot edits approved as they are and no product change in an audit of 50.
4. **Shelf pilot.** Retrain RF-DETR Nano on about 500 of the store's own photos from a few stores; gate: F1 of at least 0.85 and counting error of at most 10%, including the most crowded shelves, measured on about 180 hand-counted photos so that F1 is known to within about ±0.02 (Section 7.4).

The two pilots would run for about eight weeks. They need about 139 store-staff hours to label 500 photos of about 143 products each (the average of my test photos) at about 7 s per box (Papadopoulos et al., 2017), or AUD 4,300 at the Retail Award rate, and about 124 hours of project time, which I estimate as eight days for the ML platform lead to retrain, deploy and monitor, five days for the content team lead to run and audit the editing pilot and two and a half days for the store operations manager (about AUD 7,600 at the office rate), plus under AUD 10 of GPU time: about AUD 12,000 in total. Step 1's privacy impact assessment adds about two days of the privacy officer's time (about AUD 930, my estimate). The AVIF switch in step 2 repays itself within a year if it takes less than about 13 days of the web platform lead's time (AUD 515 a month saved, at the office rate), and a HYPIR licence is worthwhile only below about AUD 15,000 a month (Section 5.6). Even at the lower end of its range, the editing saving repays both pilots in under two months (Section 4.6).

8 Conclusion

I evaluated eight current models for four of the store's processes with one framework: real data, the same inputs for both systems, three seeds where models are random, paired tests, measured speed and cost, pass or fail limits written into the notebooks before the reported runs (after a first version with the same models), a risk table per pair and blind ratings. RF-DETR Nano and FLUX.2 [klein] are the ones to pilot, HYPIR is the most promising upscaler but needs a licence and an automatic check, and the store should adopt AVIF rather than a learned codec. System B is the better starting point, although only pair 1's lead is statistically clear, and the main lesson is that the right model depends on the task's operating point, its licence and its deployment path as much as on any benchmark score.

The evidence has clear limits: one rater (myself), small test sets in pairs 2 to 4 (8 edits, 16 enlargement inputs and 8 product photos), only some differences surviving a Holm correction (pair 1's four measures, pair 3's PSNR and SSIM, and pair 4's Kodak difference), comparisons in which the models also differ in size, precision or training recipe, rough self-timings, one metric (pair 2's product region)

that I corrected after the runs from the saved images, and costs that leave out cameras and licence fees, for which Sections 3.6 and 5.6 give break-even values instead. That is why each recommendation is a pilot with a measurable gate rather than a rollout.

No AI Use

Declaration. I designed and carried out this evaluation myself. I chose the scenario, the four pairs of models, the metrics, the limits and the statistical tests, and wrote and ran every notebook on Kaggle. I also created the evaluation data, ran the models, rated the outputs and recorded and presented the video.

I checked and edited all of this content and take responsibility for it. No result in the report was produced by AI: every number comes from my own notebook runs and saved files, and no notebook was changed or re-run during the revision.

Evidence.

- **Platforms:** the notebooks ran on Kaggle (Tesla T4, PyTorch 2.11, 31 October 2026), and my ratings were analysed on Google Colab.
- **Data:**
    - **SKU-110K** (Kaggle copy): a fixed seed-0 split of 500 training, 100 validation and 150 test photos.
    - **Amazon Berkeley Objects:** photos picked with seed 0, with every source URL saved.
    - **Kodak:** the 24-image benchmark.
- **Code and versions:** each notebook pins its package versions and GitHub commits, and run_info.json records the date, GPU, versions, commits and seeds. Table A1 lists the reported runs.
- **Baselines and metrics:** each pair uses the baselines and validity checks justified in Table 4: bicubic in pair 3, JPEG, WebP and AVIF in pair 4, and checks against the published curves.
- **Process and version history:**
    - **Version 1 (26 October 2026):** a larger evaluation.
    - **Version 2 (31 October 2026):** one notebook per pair, re-run with the same models.
    - **Version 3 (1 November 2026):** product-pixel scoring, every pair 4 BD-rate, figure and statistics scripts.
    - **3 November 2026:** comments, saving_checks.py.
    - **Problems fixed in each version:** described in Appendix A.
- **Outputs:** every CSV, figure, image and rating file (X/Y images, key.csv, my_ratings.csv) is in the Evaluation folder, so every number in the report can be traced to a file.

References

AI Weekly. (2026). *Alibaba ships Qwen-Image-2.1: 7B DiT with native 2048x2048 and RGBA, drops Apache for research-only licence*. https://aiweekly.co/alerts/alibaba-ships-qwen-image-21-7b-dit-with-native-2048x2048-and-rgba-drops-apache

Amazon Web Services. (n.d.-a). *Amazon EC2 on-demand pricing*. https://aws.amazon.com/ec2/pricing/on-demand/

Amazon Web Services. (n.d.-b). *Amazon CloudFront pricing*. https://aws.amazon.com/cloudfront/pricing/

Australian Bureau of Statistics. (2026). *Average weekly earnings, Australia, May 2026*. https://www.abs.gov.au/statistics/labour/earnings-and-working-conditions/average-weekly-earnings-australia/latest-release

Australian Taxation Office. (n.d.). *The final SG rate increase is coming on 1 July*. https://www.ato.gov.au/businesses-and-organisations/small-business-newsroom/the-final-sg-rate-increase-is-coming-on-1-july

Ballé, J., Johnston, N., & Minnen, D. (2019). Integer networks for data compression with latent-variable models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=S1zz2i0cY7

Ballé, J., Minnen, D., Singh, S., Hwang, S. J., & Johnston, N. (2018). Variational image compression with a scale hyperprior. In *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.1802.01436

Bjøntegaard, G. (2001). *Calculation of average PSNR differences between RD-curves* (Document VCEG-M33). ITU-T SG16 Video Coding Experts Group.

Black Forest Labs. (2026). *FLUX.2 [klein] 4B* [Model card]. Hugging Face. https://huggingface.co/black-forest-labs/FLUX.2-klein-4B

Blau, Y., & Michaeli, T. (2018). The perception-distortion tradeoff. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 6228–6237). https://doi.org/10.48550/arXiv.1711.06077

Can I use. (n.d.). *AVIF image format*. https://caniuse.com/avif

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. In *Computer Vision, ECCV 2020* (pp. 213–229). Springer. https://doi.org/10.1007/978-3-030-58452-8_13

Coalition for Content Provenance and Authenticity. (n.d.). *C2PA technical specification*. https://c2pa.org/specifications/

Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence Erlbaum Associates.

Collins, J., Goel, S., Deng, K., Luthra, A., Xu, L., Gundogdu, E., Zhang, X., Yago Vicente, T. F., Dideriksen, T., Arora, H., Guillaumin, M., & Malik, J. (2022). ABO: Dataset and benchmarks for real-world 3D object understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 21126–21136). https://doi.org/10.48550/arXiv.2110.06199

Commissioner Initiated Investigation into Bunnings Group Ltd (Privacy) [2024] AICmr 230. http://www.austlii.edu.au/cgi-bin/viewdoc/au/cases/cth/AICmr/2024/230.html

Competition and Consumer Act 2010 (Cth) sch 2 (Australian Consumer Law). https://www.legislation.gov.au/C2004A00109/latest

Copyright Act 1968 (Cth). https://www.legislation.gov.au/C1968A00063/latest

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems* (Vol. 36). https://doi.org/10.48550/arXiv.2305.14314

Eriksson, M., Purificato, E., Noroozian, A., Vinagre, J., Chaslot, G., Gómez, E., & Fernández-Llorca, D. (2025). Can we trust AI benchmarks? An interdisciplinary review of current issues in AI evaluation. *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society, 8*(1), 850–864. https://doi.org/10.1609/aies.v8i1.36595

Exchange-Rates.org. (2026). *AUD to USD exchange rate history for 1 October 2026*. https://www.exchange-rates.org/exchange-rate-history/aud-usd-2026-10-01

Fair Work Act 2009 (Cth). https://www.legislation.gov.au/C2009A00028/latest

Fair Work Ombudsman. (2026). *Pay guide: General Retail Industry Award [MA000004]* (effective 1 July 2026). https://calculate.fairwork.gov.au/Download/AwardSummary?awardCode=ma000004&fileType=pdf

fal. (2024). *AuraSR-v2* [Model card]. Hugging Face. https://huggingface.co/fal/AuraSR-v2

Feng, C., Zhong, Y., Gao, Y., Scott, M. R., & Huang, W. (2021). TOOD: Task-aligned one-stage object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 3510–3519). https://doi.org/10.1109/ICCV48922.2021.00349

Franzen, R. (n.d.). *Kodak lossless true color image suite*. https://r0k.us/graphics/kodak/

Gal, R., Patashnik, O., Maron, H., Bermano, A. H., Chechik, G., & Cohen-Or, D. (2022). StyleGAN-NADA: CLIP-guided domain adaptation of image generators. *ACM Transactions on Graphics, 41*(4), Article 141. https://doi.org/10.1145/3528223.3530164

General Retail Industry Award 2020 (MA000004). https://www.fwc.gov.au/documents/modern_awards/pdf/ma000004.pdf

geogeo28. (2025, November 14). *Error 404 https://huggingface.co/stabilityai/stable-diffusion-2* [Online forum post]. Hugging Face Forums. https://discuss.huggingface.co/t/error-404-https-huggingface-co-stabilityai-stable-diffusion-2/170425

Goldman, E., Herzig, R., Eisenschtat, A., Goldberger, J., & Hassner, T. (2019). Precise detection in densely packed scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 5227–5236). https://doi.org/10.48550/arXiv.1904.00853

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. In *Advances in Neural Information Processing Systems* (Vol. 27). https://doi.org/10.48550/arXiv.1406.2661

Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems* (Vol. 33, pp. 6840–6851). https://doi.org/10.48550/arXiv.2006.11239

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*(2), 65–70.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2106.09685

Hugging Face. (2026). *Qwen-Image 2.1* [Diffusers documentation]. https://huggingface.co/docs/diffusers/main/api/pipelines/qwenimage21

International Organization for Standardization. (2025). *Information technology, JPEG AI learning-based image coding system, Part 1: Core coding system* (ISO/IEC Standard No. 6048-1:2025). https://www.iso.org/standard/88911.html

JaidedAI. (n.d.). *EasyOCR* [Computer software]. GitHub. https://github.com/JaidedAI/EasyOCR

Kang, M., Zhu, J.-Y., Zhang, R., Park, J., Shechtman, E., Paris, S., & Park, T. (2023). Scaling up GANs for text-to-image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 10124–10134). https://doi.org/10.48550/arXiv.2303.05511

Ke, J., Wang, Q., Wang, Y., Milanfar, P., & Yang, F. (2021). MUSIQ: Multi-scale image quality transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 5148–5157). https://doi.org/10.48550/arXiv.2108.05997

Kelly, H. (2015, November 10). *Why is this robot in the grocery store?* CNN Money. https://web.archive.org/web/20151114074341/http://money.cnn.com/2015/11/10/technology/robot-tally-store/

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. In *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.1312.6114

Ledig, C., Theis, L., Huszár, F., Caballero, J., Cunningham, A., Acosta, A., Aitken, A., Tejani, A., Totz, J., Wang, Z., & Shi, W. (2017). Photo-realistic single image super-resolution using a generative adversarial network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4681–4690). https://doi.org/10.1109/CVPR.2017.19

Li, Y., Zhang, H., Li, L., & Liu, D. (2025). Learned image compression with hierarchical progressive context modeling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. https://doi.org/10.48550/arXiv.2507.19125

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In *Computer Vision, ECCV 2014* (pp. 740–755). Springer. https://doi.org/10.1007/978-3-319-10602-1_48

Lin, X., Yu, F., Hu, J., You, Z., Shi, W., Ren, J. S., Gu, J., & Dong, C. (2025). *Harnessing diffusion-yielded score priors for image restoration* (arXiv:2507.20590). arXiv. https://doi.org/10.48550/arXiv.2507.20590

Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). Flow matching for generative modeling. In *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2210.02747

Liu, X., Gong, C., & Liu, Q. (2023). Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2209.03003

Lu, J., Zhang, L., Zhou, X., Li, M., Li, W., & Gu, S. (2025). Learned image compression with dictionary-based entropy model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. https://doi.org/10.48550/arXiv.2504.00496

Minnen, D., Ballé, J., & Toderici, G. D. (2018). Joint autoregressive and hierarchical priors for learned image compression. In *Advances in Neural Information Processing Systems* (Vol. 31). https://doi.org/10.48550/arXiv.1809.02736

National Artificial Intelligence Centre. (2025). *Guidance for AI adoption*. Department of Industry, Science and Resources. https://www.industry.gov.au/publications/guidance-for-ai-adoption

NVIDIA. (2019). *NVIDIA T4 tensor core GPU* [Datasheet]. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf

Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., . . . Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*. https://doi.org/10.48550/arXiv.2304.07193

Papadopoulos, D. P., Uijlings, J. R. R., Keller, F., & Ferrari, V. (2017). Extreme clicking for efficient object annotation. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 4930–4939). https://doi.org/10.48550/arXiv.1708.02750

Peebles, W., & Xie, S. (2023). Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 4195–4205). https://doi.org/10.48550/arXiv.2212.09748

Privacy Act 1988 (Cth). https://www.legislation.gov.au/C2004A03712/latest

Qwen Team. (2025). *Qwen-Image technical report* (arXiv:2508.02324). arXiv. https://doi.org/10.48550/arXiv.2508.02324

Qwen Team. (2026). *Qwen-Image-2.1* [Model card]. Hugging Face. https://huggingface.co/Qwen/Qwen-Image-2.1

Racklify. (n.d.). *How much does product image retouching cost? Pricing models and budgeting for e-commerce*. https://racklify.com/encyclopedia/how-much-does-product-image-retouching-cost-pricing-models-and-budgeting-for-ecommerce/

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 779–788). https://doi.org/10.1109/CVPR.2016.91

Robinson, I., Robicheaux, P., Popov, M., Ramanan, D., & Peri, N. (2026). RF-DETR: Neural architecture search for real-time detection transformers. In *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2511.09554

Roboflow. (2026, May 6). *RF-DETR vs. alternatives: Benchmarks and deployment compared* [Blog post]. https://roboflow.com/blog/rf-detr-vs-alternatives

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 10684–10695). https://doi.org/10.48550/arXiv.2112.10752

Ruiz, N., Li, Y., Jampani, V., Pritch, Y., Rubinstein, M., & Aberman, K. (2023). DreamBooth: Fine tuning text-to-image diffusion models for subject-driven generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 22500–22510). https://doi.org/10.48550/arXiv.2208.12242

Sapkota, R., Cheppally, R. H., Sharda, A., & Karkee, M. (2025). *YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection* (arXiv:2509.25164). arXiv. https://doi.org/10.48550/arXiv.2509.25164

Sharma, G., Wu, W., & Dalal, E. N. (2005). The CIEDE2000 color-difference formula: Implementation notes, supplementary test data, and mathematical observations. *Color Research & Application, 30*(1), 21–30. https://doi.org/10.1002/col.20070

Squareshot. (n.d.). *Per-image product photography pricing*. https://www.squareshot.com/post/per-image-pricing

Stability AI. (2022). *Stable Diffusion v2-1-base model card* [Model card; copy of stabilityai/stable-diffusion-2-1-base, revision 0094d48]. Hugging Face. https://huggingface.co/Manojb/stable-diffusion-2-1-base

Supermarket News. (2020). *Schnuck Markets rolls out shelf-scanning robots to over half of store base*. https://www.supermarketnews.com/grocery-technology/schnuck-markets-rolls-out-shelf-scanning-robots-to-over-half-of-store-base

SupPixel Pty Ltd. (2024). *HYPIR licence* [Software licence]. GitHub. https://github.com/XPixelGroup/HYPIR

thedatasith. (n.d.). *SKU110K annotations* [Data set]. Kaggle. https://www.kaggle.com/datasets/thedatasith/sku110k-annotations

Trade Marks Act 1995 (Cth). https://www.legislation.gov.au/C2004A04969/latest

Ultralytics. (2026). *Ultralytics YOLO26* [Documentation]. https://docs.ultralytics.com/models/yolo26/

Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., & Ding, G. (2024). YOLOv10: Real-time end-to-end object detection. In *Advances in Neural Information Processing Systems* (Vol. 37). https://doi.org/10.48550/arXiv.2405.14458

Wang, X., Xie, L., Dong, C., & Shan, Y. (2021). Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops* (pp. 1905–1914). https://doi.org/10.48550/arXiv.2107.10833

Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing, 13*(4), 600–612. https://doi.org/10.1109/TIP.2003.819861

Wang, Z., Simoncelli, E. P., & Bovik, A. C. (2003). Multiscale structural similarity for image quality assessment. In *The Thirty-Seventh Asilomar Conference on Signals, Systems & Computers* (Vol. 2, pp. 1398–1402). IEEE. https://doi.org/10.1109/ACSSC.2003.1292216

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin, 1*(6), 80–83. https://doi.org/10.2307/3001968

Xiong, H., Xu, Y., Zhu, K., Wang, Y., & Ye, B. (2026). *HiDE: Hierarchical dictionary-based entropy modeling for learned image compression* (arXiv:2603.06766). arXiv. https://doi.org/10.48550/arXiv.2603.06766

Zhang, R., Isola, P., Efros, A. A., Shechtman, E., & Wang, O. (2018). The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 586–595). https://doi.org/10.48550/arXiv.1801.03924

Zhu, X., Su, W., Lu, L., Li, B., Wang, X., & Dai, J. (2021). Deformable DETR: Deformable transformers for end-to-end object detection. In *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2010.04159

Zhu, Y., Jin, T., Pruksachatkun, Y., Zhang, A., Liu, S., Cui, S., Kapoor, S., Longpre, S., Meng, K., Weiss, R., Barez, F., Gupta, R., Dhamala, J., Merizian, J., Giulianelli, M., Coppock, H., Ududec, C., Kellermann, A., Sekhon, J., . . . Kang, D. (2025). Establishing best practices in building rigorous agentic benchmarks. In *Advances in Neural Information Processing Systems* (Vol. 38). https://doi.org/10.48550/arXiv.2507.02825

Appendix A: Implementation Evidence

The 48658057 Evaluation folder holds one folder per pair with the executed notebook and a results folder with the CSV files, figures and rating files (X and Y images, key.csv and my_ratings.csv) it wrote. README.md explains how to re-run each notebook on Kaggle and gives an implementation statement for each pair. Table A1 lists the reported runs.

Table A1

Reported Runs (Kaggle, Tesla T4, PyTorch 2.11, 31 October 2026)

Pair	Notebook	Main versions	Run time
1	pair1_shelf_detection	ultralytics 8.4.189, rfdetr 1.11.2	58.8 min
2	pair2_image_editing	diffusers 0.41.0, transformers 5.20.1	66 min
3	pair3_super_resolution	diffusers 0.41.0, pyiqa 0.1.16, HYPIR commit b61d107	18.7 min
4	pair4_image_compression	compressai 1.2.8, imagecodecs 2026.1.14, HPCM commit c271cfe, DCAE commit e2525a0	18.3 min

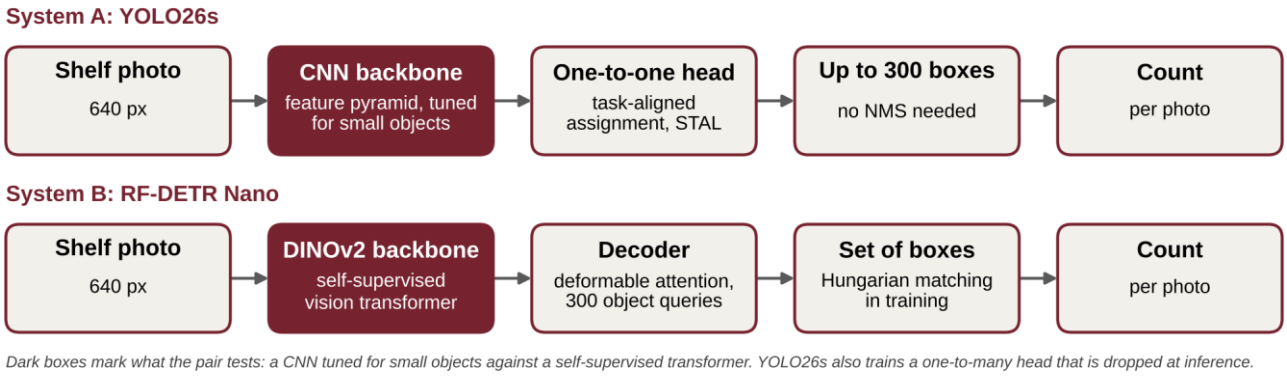
Note. The last cell of each notebook, added after the Kaggle run, analyses my ratings (mean score, share rated 4 or 5, Wilcoxon test and review time); I ran it on Google Colab with the rating files. Pair 2 also has product_pixel_check.py, which re-scores SSIM on product pixels from the saved images; report_figures.py redraws Figures 2 to 6 from the results folders; and statistics_check.py computes the Holm-corrected p, bootstrap intervals, effect sizes, the shelf-density subgroups and trend, the Spearman correlations and the per-photo BD-rates against AVIF. saving_checks.py gives the 95% intervals for the share of outputs I rated publishable, how the shelf saving changes with the manual check time, how many products an unchecked RF-DETR Nano count misses and the HYPIR saving after the automatic gate. Only the seed 0 output images of pair 3 are included to keep the folder small; the scores of every seed are in the CSV files.

Version history and problems fixed. Version 1 (26 October 2026) was a much larger evaluation that went far beyond the assignment's scope. Version 2 (31 October 2026) re-scoped it to one shorter notebook per pair with the same models and re-ran every pair; only my 50.5 s manual shelf-check timing comes from version 1. Version 3 (1 November 2026) followed a re-check of the results: I found that pair 2's product scores used the rectangle around the product and added product_pixel_check.py, reported every pair 4 BD-rate, added scripts that redraw the report's figures and compute Holm-corrected tests, bootstrap intervals and effect sizes, toned down claims that rested on small or confounded comparisons and corrected how the densest shelf photo's detections were described. On 3 November 2026 I added comment lines and section headings to the notebooks for readability, without re-running them; no code or output changed. The same day I added saving_checks.py, which gives the uncertainty of the savings in pairs 2 and 3, the sensitivity of the shelf saving to the manual check time, the products a shelf count misses and the HYPIR saving after the automatic gate. During the version 2 runs, the Kaggle copy of SKU-110K used YOLO label files, so I made the loader read both formats; the first run of pair 2 stalled while loading Qwen-Image-2.1, so I split the models into separate cells with progress lines and re-ran it; and in pairs 3 and 4 the first rating item, a white easel on a white background, cropped to a blank patch, so I rated the other seven. **Data.** SKU-110K (research use) through a Kaggle copy, Amazon Berkeley Objects photos (CC BY 4.0) with every source URL saved in the results, and the Kodak images; no personal data was collected.

Appendix B: Model Architectures

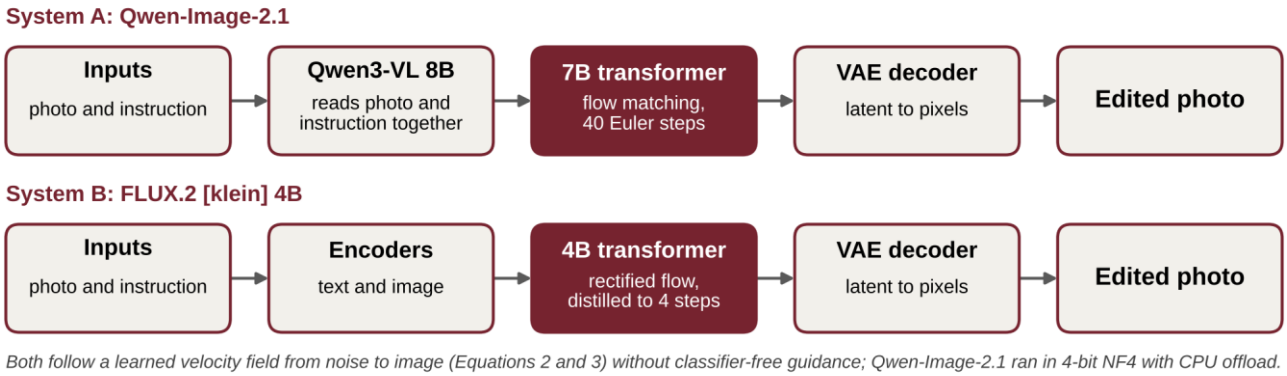
Figures B1 to B4 show each pair's pipeline from input to output. Dark boxes mark the core technique that the pair compares.

Figure B1
Pair 1: YOLO26s and RF-DETR Nano



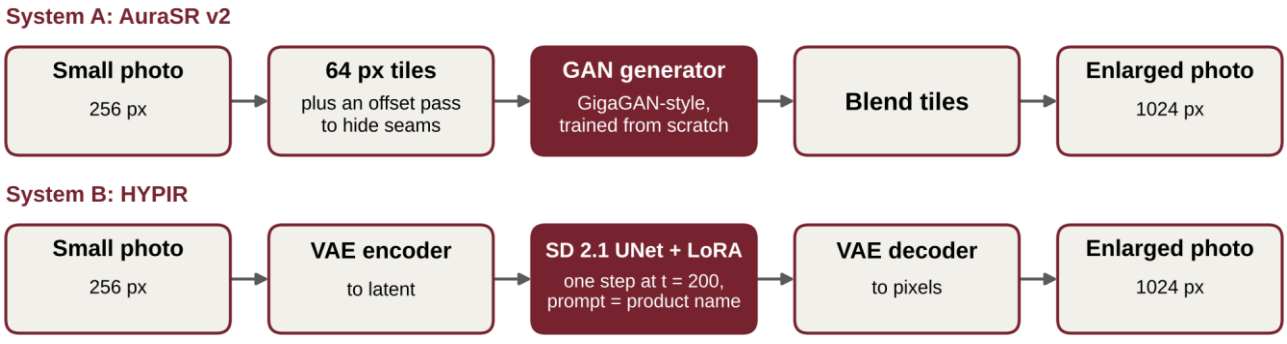
Note. Own diagram after Ultralytics (2026) and Robinson et al. (2026).

Figure B2
Pair 2: Qwen-Image-2.1 and FLUX.2 [klein] 4B



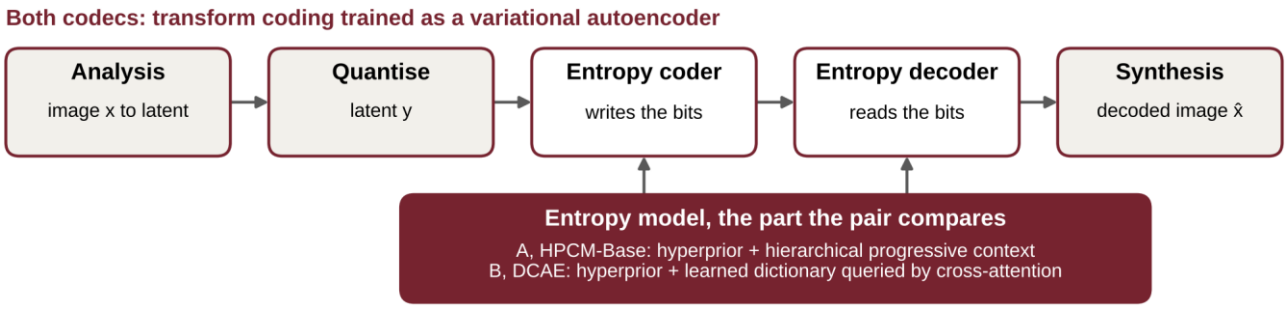
Note. Own diagram after Qwen Team (2026), Hugging Face (2026) and Black Forest Labs (2026).

Figure B3
Pair 3: AuraSR v2 and HYPIR



Note. Own diagram after fal (2024) and X. Lin et al. (2025).

Figure B4
Pair 4: HPCM-Base and DCAE



The coder and decoder must get bit-identical probabilities from the entropy model; this is what failed across devices (Section 6.4).

Note. Own diagram after Ballé et al. (2018), Li et al. (2025) and Lu et al. (2025).